CAPSTONE 02 / DAYS 1 AND 2

Settlement Intelligence

Bank of New York

The business case, the code, the release, and the rollback

Worked examples, operating commands, evidence, and panel preparation.

Contents

01 The settlement problem 3

02 Day 1 deliverables and architecture 4

03 Source files, keys and time 5

04 Bronze, Silver and Gold 6

05 Incremental ingestion and data quality 7

06 KPIs and worked calculations 8

07 API and dashboard 9

08 Running the application and testing it 10

09 Day 2 production readiness and security 11

10 GitLab CI, Jenkins and release artifacts 12

11 Docker and environment design 13

12 Blue-green deployment and cutover 14

13 Incident response and rollback 15

14 PowerShell runbook for the live demonstration 16

15 Section 12 fault scenarios from the brief 18

16 Permanent controls and operations 20

17 Leadership presentation and peer review 21

18 Panel questions and answers 22

Appendix A / Requirements as supplied 24

Appendix B / Repository source and runbooks 41

Appendix C / Evidence and repository 76

01 The settlement problem

The Bank of New York case begins with ₹48 crore in reported transactions and ₹45.8 crore settled at the reporting cutoff. The ₹2.2 crore difference is an investigation queue. A nightly spreadsheet can show the difference without identifying the payments behind it, whether matching settlement records arrived later, or which merchants should be investigated first.

A payment attempt, a successful payment, and a completed settlement describe different stages. A gateway can confirm payment success while the processor is still preparing the merchant settlement. A later payout or settlement record may arrive in parts. Before comparing totals, align the date range, currency, eligible statuses and the point in time at which the source was captured. The case does not establish that the whole ₹2.2 crore is caused by latency, fraud, or a loss.

Why ACID does not settle this question

An ACID transaction keeps operations inside its database transaction consistent. The customer debit, payment confirmation, merchant payout and reporting export can belong to separate systems and times. Their states must be reconciled. A spreadsheet or dashboard can show a gap while the underlying payout is pending or while an export is incomplete.

Who uses the platform

The Head of Payments tracks volume, settlement rate, gap, SLA and merchant risk. An operations analyst filters by payment date or merchant, follows the daily trend, prioritizes the largest gaps and opens individual exceptions. A data engineer monitors batch quality and traces a number back to raw rows. The bank or its payment processor handles money movement and case resolution.

Bank totals → merchant gaps → individual payment exceptions → source records

The sample-data baseline is a different scope from the case headline. Our January 1 to October 1 run recorded 31,763 successful payments, ₹13,66,75,776.64 in payment value, ₹12,94,29,359.85 credited, and ₹72,46,416.79 outstanding. The rate was 94.70%. Do not present the sample run as reconciliation of the fictional ₹2.2 crore.

02 Day 1 deliverables and architecture

How the tasks fit together

Task	What it asked for	Repository evidence
1. Specification	Business rules, technical design and Gherkin acceptance scenarios	docs/business.md, docs/technical.md, tests/acceptance.feature
2. Model and pipeline	Physical SQLite model, keys, constraints, staged ingestion and incremental updates	settlement/schema.sql, settlement/db.py, settlement/ingest.py
3. Application	FastAPI contract, browser dashboard and full test suite	settlement/api.py, settlement/web, tests/test_platform.py
4. Engineering review	Tests, security controls and DEV / TEST / PROD deployment design	tests, Dockerfile, docs/deployment.md, CI configuration

Source CSVs → Python ingestion → Bronze → Silver → Gold → FastAPI → HTML / JavaScript

The full-stack deliverable is the dashboard and API in Task 3, powered by the model and pipeline from Task 2. Python's csv module reads records, sqlite3 runs SQL and transactions, FastAPI handles HTTP and OpenAPI, and Pydantic validates typed response models. The application-specific logic decides which rows to admit, which transactions to refresh, when a payment completed and how to reconcile it.

A single SQLite file contains separate tables for the logical layers. No manual upload button or scheduled watcher is required by this implementation. An operator invokes the batch CLI with a folder. An automated workflow could call the same CLI when files land. The API reads SQLite after ingestion commits. Refreshing the dashboard fetches the new aggregates.

03 Source files, keys and time

CSV	One row represents	Fields
transactions.csv	A payment attempt	transaction_id, merchant_id, customer_id, transaction_ts, amount, currency, status, payment_channel
settlements.csv	A settlement record	settlement_id, transaction_id, settlement_ts, settlement_amount, settlement_status, settlement_batch
merchant.csv	A merchant risk profile for an effective period	merchant_id, merchant_name, merchant_category, country, risk_level, effective_from, effective_to
payment_events.csv	One payment lifecycle event	event_id, transaction_id, event_type, event_ts, ingestion_ts, processing_ms

The supplied dataset contains one base batch and an incremental batch with the shared suffix 2026-10-01. Each batch supplies all four files. A consistent suffix prevents mixing files from unrelated deliveries. Filenames tell us which files go together; a digest of the batch identifies an exact replay.

Transaction time places a payment in the dashboard's reporting period. Settlement time measures when settlement occurred. Event time tells us when the lifecycle event happened. Ingestion time tells us when the event reached the data system. Filtering on ingestion time can move an older payment into a newer report even when the underlying transaction date never changed.

Historical merchant risk

Suppose M100 was LOW in February, HIGH in May, and MEDIUM from July. A February payment retains LOW even if the merchant's current status is MEDIUM. Merchant summary rows display the most recent successful payment's risk within the selected scope. Merchant history intervals have inclusive end dates. A blank end date is open; overlapping periods create ambiguity and go to quarantine.

04 Bronze, Silver and Gold

Layer	Physical model	Purpose
Bronze	batch, bronze, finding and retry support	Track delivered batches and preserve source payload, row number and provenance. Log data issues and retry eligible unresolved references.
Silver	merchant, payment, settlement, event	Validate types and business keys, enforce references, preserve historical risk and keep each fact at its native grain.
Gold	gold	Maintain one row per successful payment, with amount, actual and credited settlement, completion, SLA and merchant attributes.

Why grain is the first design question

Grain is what one row represents. A payment has one transaction ID but may have several settlement IDs and lifecycle event IDs. Merchant history has a composite key of merchant and effective start. A direct join to both settlement and event facts can multiply rows. For example, two settlement rows and three event rows could yield six joined records for one payment. Aggregating payment-level measures after that join can count the original value six times.

The application uses the successful payment as the reporting grain. A natural day column supports date grouping without requiring a standalone date dimension for this capstone. Customer IDs are retained only in raw payloads, avoiding unnecessary analytics exposure. Primary keys enforce uniqueness, foreign keys enforce valid parents where defined, CHECK constraints enforce field invariants, and indexes support lookups by transaction, day and merchant.

Walk one payment through all layers

Bronze preserves the original ₹1,000 payment and two settlement records of ₹600 and ₹400. Silver links the settlements to the payment while retaining their own IDs. Gold has one ₹1,000 payment row with ₹1,000 credited and zero gap. If the ₹400 row has not arrived, Gold credits ₹600 and shows ₹400 outstanding. The API aggregates Gold over the selected date range, and the browser displays those totals.

05 Incremental ingestion and data quality

Identify batch → record raw rows → validate Silver facts → retry eligible children → refresh affected Gold → commit

The ingestion command checks the four-file batch and computes a fingerprint. It writes Bronze payloads with lineage, validates each row and collects affected payment IDs. It refreshes Gold for those IDs and commits the batch atomically. Merchant history changes may affect earlier payments, and missing-parent records can be retried when their parent arrives. A successful exact replay does not double-count facts or refresh the same batch again.

How failures are classified

Situation	Disposition	Reason
Invalid currency, negative amount, malformed timestamp or field	Reject and record finding	The row breaks the contract.
Missing parent or conflicting ID	Quarantine; retry eligible missing references	The row cannot be safely linked or applied yet.
Overlapping merchant history	Quarantine	The effective risk would be ambiguous.
Identical business record	Ignore; record duplicate-event warning where applicable	Do not count the same fact twice.
Late event delivery or excess settlement	Warn	The row or reporting condition merits investigation.
Successful payment without settlement	Business exception	The payment exists and should remain visible.

A PENDING settlement can transition to a terminal state under the same ID and payment. A completed settlement earlier than its payment is invalid. Events are ordered by event timestamp and ID. The late-arrival warning compares event and ingestion times; settlement SLA measures elapsed time from successful payment to completion. These clocks answer different questions.

Atomicity and replay

If ingestion fails, rollback avoids committing half a batch or publishing partial Gold results. Exact replay is controlled by the batch fingerprint. A distinct batch can still contain an identical business record, so uniqueness and per-record comparison matter independently. SQLite WAL supports reads during a writer transaction; API requests use a consistent read transaction. SQLite remains a single-host choice and writer concurrency must be managed.

06 KPIs and worked calculations

Measure	Calculation	Interpretation
Successful count	Count successful payment rows	Payment attempts that failed or reversed do not contribute.
Transaction volume	Sum successful payment value	Denominator for the value-based settlement rate.
Actual settled	Sum completed settlement rows for each payment	Preserves any amount above the original payment.
Credited settled	For each payment, min(actual, payment amount)	The value recognized toward the rate.
Gap	Successful value minus credited value	Outstanding credited payment value.
Settlement rate	100 × credited value / successful payment value	A value ratio, not the percentage of payments by count.
Excess	Actual settled minus credited settled	A separate anomaly signal, even if the rate looks normal.
30-minute SLA	100 × completed within 30 minutes / fully completed payments	Partial and unsettled payments are excluded from this denominator.
Merchant risk KPI	Count merchants with settlement rate below 95% AND SLA below 90%	The exception list uses OR, so it is usually broader.

Three payments, one reporting total

P101 is ₹1,000 and settles in ₹600 and ₹400, reaching full completion after 25 minutes. P102 is ₹500 and has ₹200 settled. P103 is ₹500 with no settlement. Total payment value is ₹2,000, credited settlement is ₹1,200, the gap is ₹800, and the rate is 60%. One payment is fully complete and within 30 minutes, so the SLA is 100%. If P101's final portion arrived after 35 minutes, the rate would still be 60% but the SLA would be 0%.

An exactly 30-minute completion counts as on time. A rate without a denominator is null. Merchant comparisons are strictly below their thresholds. The project uses integer paise in storage and converts to INR for API output. Daily reporting groups both successful value and credited settlement by payment date, so late settlements can change an earlier day's view.

07 API and dashboard

Endpoint	Purpose
GET /	Serve the HTML dashboard
GET /health	Check access to the Gold and batch schema
GET /docs	OpenAPI interactive documentation
GET /api/v1/data-status	Available date range and last successful ingestion
GET /api/v1/settlement-summary	Counts, monetary totals, rates, gap, risk and excess
GET /api/v1/daily-trend	Daily payment and credited settlement value
GET /api/v1/merchant-gaps	Top 10 merchants ranked by outstanding value
GET /api/v1/merchant-exceptions	Merchants below the settlement or SLA threshold
GET /api/v1/payment-exceptions	Outstanding or late payments, with paging

The reporting endpoints take inclusive start_date and end_date parameters in YYYY-MM-DD and an optional merchant_id. A valid authorized request returns 200. Malformed dates or reversed ranges return 400. An unknown merchant returns 404. Missing parameters or a merchant ID that violates the pattern return 422. Invalid or missing bearer credentials return 401. Missing server token configuration or an unavailable database can return 503.

The browser sends the operations token in the Authorization header after the user enters it. JavaScript retains the token in memory, calls the same-origin API, and renders KPI cards, daily values, top merchant gaps and exception tables. It can drill from a merchant into its payment exceptions. The API exposes excess_settled_amount; the dashboard version in the drill did not render a card for it.

Token handling in tests and deployment

The test fixture supplies a controlled token in the temporary test process and sends it with TestClient. This tests the real authorization code without needing the running app or a production secret. The dashboard sends the header on each API request after entry. Nginx forwards the request and the application checks it. The proxy and docker start do not generate a token.

Parameterized SQL binds user values using placeholders. Fixed internal SQL fragments select filters and allowlisted identifiers. The query connection is read-only at the SQLite statement level. Customer IDs are not returned by the API, access logs are disabled in the launch command, and API responses have no-store caching. For a real deployment use TLS, restricted ingress and managed secrets, with individual identity and auditing where required.

08 Running the application and testing it

Local run in PowerShell

```
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements-dev.txt
$data = 'C:/Users/PARTHA/OneDrive/Desktop/capstone2_datasets'
.venv/Scripts/python.exe -m settlement.ingest $data
.venv/Scripts/python.exe -m settlement.ingest "$data/incremental"
.\start.ps1
```

Open http://127.0.0.1:8000 and enter the operations token shown locally. Both the ingestion command and API use var/settlement.db by default. SETTLEMENT_DB can point both processes at another file. Re-run the incremental batch to check replay behavior, then refresh filters in the browser.

Six automated tests

Test focus	What it proves
Silver grain, quality and risk	Unique grain, historical risk and visibility of data quality issues
Incremental split, orphan repair and replay	A late child can reconcile when its parent arrives; split rows and reruns do not inflate results
Atomic batch failure	A broken batch does not expose half-written results
API contract and dashboard	Response codes, authorization, dashboard HTML and response data
Over-settlement cap and late SLA	Excess stays separate and completion timing drives SLA
Merchant gap ranking	Correct aggregation and order in the ranked merchant output

The suite makes tiny CSV fixtures during the run and uses a temporary SQLite database. The API client queries that temporary database so assertions are deterministic. A failing test cannot modify the real demo database. The dataset validation and load timing checks are separate from the six fixture tests.

```
.venv/Scripts/python.exe -m pytest -v
.venv/Scripts/python.exe scripts/day2_readiness.py
```

The mandatory split-settlement regression used three payments and checked ₹30,000 successful volume, ₹20,000 credited, ₹10,000 gap and 66.67% settlement rate. An independent Silver calculation matched the full-scope API summary and the merchant exception IDs. Foreign-key violations were zero.

09 Day 2 production readiness and security

The brief calls for all required tests passing, zero critical findings, HTTP 200 health, under 500 ms under normal load, verified deployments, reconciliation, evidence and rollback. Local load validation made 100 successful requests using five concurrent workers. The summary p95 was 387.308 ms, max 415.358 ms. Merchant exceptions p95 was 362.666 ms, max 387.606 ms. These figures describe that workload and host.

Gate	Tool and observed result	What to explain
Static analysis	Bandit 1.9.4; no unresolved findings	Fixed SQL fragments and parameter binding explain the reviewed suppressions.
Secret scan	Gitleaks 8.30.1; two hash alerts in the original workspace scan, both false positives	The workspace scan had a limited scope. The Jenkins job also ran a Git scan.
Dependencies	pip-audit 2.10.1; 15 packages checked, zero known findings in that run	This is a point-in-time check of resolved dependencies.
Container scan	Trivy; 0 critical, 51 high, 81 medium, 67 low, 19 unknown in the initial scan	The zero-critical gate passed. High findings remain open and should be tracked.

The image scan covers base OS packages and runtime tooling in addition to application dependencies. A scan passing its gate does not certify the container for enterprise production. Follow-up includes fixed base packages, removing unneeded build tools, rebuilding and rescanning. Preserve report severity, package, installed version, fixed version, remediation and owner.

Evidence map for Day 2

Required evidence	Where to find it or current status
01 Test results	evidence/01_test_results, local logs, JUnit, KPI and timing JSON
02 Security results	evidence/02_security_results, local scanner reports and assessment
03 GitLab pipeline	.gitlab-ci.yml and runner instructions. Account verification blocked a verified run.
04 Jenkins pipeline	Jenkinsfile, build #2 log in GitLab and #3 console in Jenkins
05 Docker image	Jenkins image.json and capstone-image.tar artifacts, plus Docker image tags
06 Deployment version	Cutover report maps Blue #2 and Green #3 to versions and ports
07 Blue-green status	Same URL, deployment header before and after reload, Blue retained
08 Smoke tests	DEV JSON and endpoint status and timings in cutover report
09 KPI reconciliation	Independent Silver baseline and the before/after comparison

10 GitLab CI, Jenkins and release artifacts

GitLab repository → GitLab CI primary design / Jenkins verified build path → scanned image → deployment

GitLab hosts the code. GitLab CI evaluates .gitlab-ci.yml and queues jobs for an eligible runner. A registered runner is an executor, so registration by itself does not prove a pipeline ran. Docker Desktop gives the runner a Docker engine. Jenkins is a separate CI service that can pull the same repository and build independently. Enterprises may maintain it for legacy agents, plugins, credentials and existing pipelines while migrating or supporting several teams.

GitLab stage	Purpose
validate	Compile Python, check the shell script and verify the Dockerfile exists
test	Install dependencies and run pytest, publishing JUnit XML
security	Run SAST, secret scan and dependency scan
build	Build and push a uniquely tagged candidate to the project registry
package	Scan the candidate by digest and publish the approved release reference
deploy_dev	Deploy the approved image to DEV with its own configuration and data
smoke_test	Check deployed image identity, health, JSON and settlement reconciliation
deploy_prod	Manual controlled deployment after upstream checks pass

rules restrict jobs to the protected default branch; needs declares upstream dependencies; artifacts preserve test and security evidence; cache reduces repetitive downloads. Environment-scoped protected secrets distinguish DEV and PROD. In this project a GitLab.com verification prompt prevented evidence of a completed hosted run. The Docker runner did register. Do not claim Jenkins success as GitLab CI success.

Jenkins execution

The capstone-day2 Pipeline job uses Pipeline script from SCM, the GitLab repository, main, and Jenkinsfile. The Windows agent is labeled capstone-windows, with Docker Desktop in Linux mode. It checks out the commit, installs dependencies, tests, scans, builds and packages. Build #1 failed at the unavailable Get-FileHash command after its tests and scan ran. The pipeline was fixed to compute SHA-256 using .NET and verified against the archive. Build #2 succeeded in 5m 46s. Build #3 succeeded for the Version 2 label at commit 53c1ddf.

An image tag is a human-friendly name, while the image ID or registry digest identifies content in its respective context. capstone-image.tar is a saved image archive; SHA-256 verifies the archive bytes. Jenkins archived the reports and image. Use docker load -i capstone-image.tar on a host with the archive. Build metadata had a null commit field in the observed report, so the checkout line in Jenkins is the source for the build commit.

11 Docker and environment design

The Dockerfile starts from Python 3.11 slim, installs dependencies, copies the settlement application and runs Uvicorn as UID 10001. It expects SQLite at /data/settlement.db. The mounted data volume is separate from the image. Changing data in that volume changes the next query's result without rebuilding the image. A named volume persists across container replacement.

Object	What it is	Example
Image	Packaged app and dependencies	capstone-settlement:jenkins-2
Container	A running or stopped instance created from an image	capstone-blue
Volume	Persistent data mounted into a container	capstone-blue-data mounted at /data
Proxy	Separate Nginx container sharing the app network	capstone-dashboard-proxy
Registry	Remote image storage and distribution	GitLab project Container Registry

docker run takes an image and creates a container. docker start takes an existing container name or ID and starts it. docker ps -a includes stopped containers. docker port lists host mappings. Run those commands in Windows PowerShell or the VS Code terminal, with Docker Desktop running. Docker Desktop's Exec tab runs commands inside the selected container.

```
docker ps -a
docker image ls
docker port capstone-dashboard-proxy
docker start capstone-blue
docker start capstone-dashboard-proxy
curl.exe -i http://127.0.0.1:8080/health
docker exec capstone-blue printenv SETTLEMENT_API_TOKEN | Set-Clipboard
```

The local app starts as the container's main process. An existing running container generally just needs its URL opened. A container ID passed to docker run produces a No such image error. If Blue has no host port mapping, use the Nginx proxy. The browser cannot reach Blue's internal port through Windows localhost:8000 unless a host mapping or another server exists there.

DEV, TEST and PROD

DEV runs the app against an approved development volume. The manual DEV demo used jenkins-2, capstone-development-data and port 8081. TEST verifies authentication, filters, replay and atomic failure against its own data. PROD uses approved data, a managed token and ingress controls. A release should promote the same scanned image digest between environments, while each environment supplies its own data and secrets. The local Docker Desktop blue-green exercise simulates production routing; it does not establish enterprise deployment readiness.

12 Blue-green deployment and cutover

Browser localhost:8080 → host port mapping → Nginx port 80 → active app port 8000

Blue V1 ran from jenkins-2 and supplied the known-good baseline. Green V2 ran from jenkins-3. We added the Version 2 label, committed and pushed it, ran Jenkins again, and started Green with its own SQLite snapshot. Green's preview maps host port 8083 directly to its internal port 8000. Blue's internal 8000 has no direct host mapping. Both app containers can listen on 8000 because they have separate network namespaces.

The Nginx container is on the same Docker network and resolves capstone-blue and capstone-green by name. It routes all requests to the configured upstream. Starting Green does not change that upstream. Changing default.conf, validating it and reloading Nginx changes the shared URL's destination. Blue stays available for rollback.

Live demonstration and validation

1. Open port 8080 while Blue is active. Verify the blue-v1 header and baseline.

2. Open Green on 8083. Verify Version 2, database-backed health, the API response and reconciliation against Blue.

3. Change Nginx from capstone-blue:8000 to capstone-green:8000 and set the green-v2 header. Run nginx -t, reload and wait for the expected header.

4. Refresh the same port 8080 tab. Confirm Version 2 and repeat API checks through the shared route.

5. Keep Blue available. In rollback, reverse the target and header, reload and validate business results again.

```
docker exec capstone-dashboard-proxy nginx -t
docker exec capstone-dashboard-proxy nginx -s reload
curl.exe -i http://127.0.0.1:8080/health
```

Post-cutover check	Observed
/health	HTTP 200, green-v2 header, 83 ms
/api/v1/settlement-summary	HTTP 200, 338 ms; 31,763 transactions and 94.70%
/api/v1/merchant-exceptions	HTTP 200, 161 ms; 61 exceptions

The measured dataset period was 2026-01-01 through 2026-10-01. SLA was 82.03%, merchant risk count was 16, and excess was zero. After the cutover, all measured smoke requests were below 500 ms. A reload command returning success is followed by an HTTP check because the old worker can still answer briefly. A one-button pipeline could validate Green, require approval, apply and reload the route, then verify the same endpoint. The observed switch was manual.

13 Incident response and rollback

What we actually ran

We took a SQLite backup of Green and simulated duplicate settlement reporting by doubling actual on three fully settled Gold payments belonging to three merchants. Credited stayed at the payment value. Green returned HTTP 200 from health and summary. Its capped rate stayed 94.70%, but excess rose to ₹5,60,042.46. No source data was changed in Blue. The fault was in local reporting data and did not move money.

Phase	Deployment	Settlement rate	Excess	Evidence
Clean after cutover	Green V2	94.70%	₹0	Cutover and baseline evidence
Injected fault	Green V2	94.70%	₹5,60,042.46	Authenticated summary; fault captured on Green preview
After route rollback	Blue V1	94.70%	₹0	Shared URL, blue-v1 header, HTTP 200
Green data cleanup	Green preview	94.70%	₹0	Restored changed rows from backup and rechecked

Why roll back?

The dashboard supports merchant investigation. When reporting values cannot be trusted, operations can prioritize the wrong cases. Blue was healthy with a validated baseline and an isolated database. Switching the shared route restored trusted reporting while Green was inspected. In the recorded incident, rollback occurred at about 17:58 IST on 12 November 2026. The first probe was too early after reload; a follow-up showed blue-v1. The faulty Green response was captured via the preview after rollback, then the three modified values were restored.

How to show it

Start on clean Green through port 8080. Show the header, rate and excess before injection. In the isolated local drill, back up Green, change three known test rows and query the authenticated summary. The visual Version 2 label proves routing; PowerShell/API output proves excess because the UI does not show the metric. Route back to Blue, show HTTP 200, blue-v1 and zero excess through 8080. Then restore the Green rows and verify the preview. If presenting from evidence rather than rerunning, label the source and timing of each captured response.

14 PowerShell runbook for the live demonstration

Use the local demo containers and their isolated volumes. Before opening the shared URL, start Blue and Nginx if they are stopped. Start Green for the preview. The same operations token was set on both app containers during the demo. Run the following from the project directory.

```
docker start capstone-blue capstone-green capstone-dashboard-proxy
curl.exe -i http://127.0.0.1:8080/health
curl.exe -i http://127.0.0.1:8083/health
docker exec capstone-blue printenv SETTLEMENT_API_TOKEN | Set-Clipboard
```

Paste the token into the dashboard. Before switching traffic, compare the authorized summary on 8080 with the same request on 8083. Use the same date range throughout. For a clean baseline, expect 94.70% and zero excess. The X-Capstone-Deployment header appears on the shared URL; the direct preview does not go through Nginx.

```
$token = docker exec capstone-blue printenv SETTLEMENT_API_TOKEN
$headers = @{Authorization = "Bearer $token"}
$query = 'start_date=2026-01-01&end_date=2026-10-01'
$route = 'http://127.0.0.1:8080'
$reply = Invoke-WebRequest -UseBasicParsing -Headers $headers `
  -Uri "$route/api/v1/settlement-summary?$query"
$reply.Headers['X-Capstone-Deployment']
$reply.Content | ConvertFrom-Json | Select-Object settlement_rate, excess_settled_amount
```

To switch to Green, open var/blue-green/default.conf in the project folder. Change capstone-blue:8000 to capstone-green:8000 and blue-v1 to green-v2. Keep a copy of the last good configuration. Validate with nginx -t before reloading. Restore the old configuration if validation fails. After reload, check the header and the Version 2 label on 8080. Check summary and merchant exceptions as well as health. For rollback, reverse both replacements and repeat the same checks.

Reproducing the observed data-only fault

Run the following only on the isolated Green demo database after verifying it is clean. The script saves a consistent backup and changes actual settlement for three fully credited payments from different merchants. Blue uses a separate volume. Show excess through the API before rolling back.

```
@'
import sqlite3
from contextlib import closing
from uuid import uuid4

with closing(sqlite3.connect("/data/settlement.db")) as db:
    backup_path = f"/data/pre-demo-{uuid4().hex}.db"
    with closing(sqlite3.connect(backup_path)) as backup:
        db.backup(backup)
    with db:
        db.execute("BEGIN IMMEDIATE")
        excess = db.execute("SELECT SUM(actual - credited) FROM gold").fetchone()[0]
        if excess != 0:
            raise SystemExit("Expected a clean Green baseline")
        chosen = {}
        for payment, merchant, amount in db.execute("""
            SELECT id, merchant_id, amount FROM gold
            WHERE actual = amount AND credited = amount
              AND day BETWEEN '2026-01-01' AND '2026-10-01'
            ORDER BY amount DESC, id
        """):
            chosen.setdefault(merchant, (amount, payment))
            if len(chosen) == 3:
```

```
                break
        if len(chosen) != 3:
            raise SystemExit("Expected three eligible merchants")
        db.executemany("UPDATE gold SET actual = actual + ? WHERE id = ?",
                       chosen.values())
    print(backup_path)
'@ | docker exec -i capstone-green python -
```

Once port 8080 is back on Blue and its KPIs are verified, restore Green from the exact backup printed above. Pause any Green ingestion before restoring. Enter the backup path when prompted. Rerun the settlement check against port 8083 afterward.

```
$backup = Read-Host 'Enter the Green backup path printed during injection'
@'
import sqlite3
import sys
from contextlib import closing

with closing(sqlite3.connect("file:" + sys.argv[1] + "?mode=ro", uri=True)) as source:
    with closing(sqlite3.connect("/data/settlement.db")) as target:
        source.backup(target)
'@ | docker exec -i capstone-green python - $backup
```

The dashboard can show Version 2 during the fault, but its current cards do not show excess. Keep the PowerShell response visible. The four Section 12 fault scenarios use different mechanisms, described next.

15 Section 12 fault scenarios from the brief

The Day 2 incident example shows 95.2% before the issue and 103.7% during it, with HTTP 200. The brief says exact figures should come from the trainer dataset. The logged local drill did not execute any of the four listed faults or display a rate above 100%. For the executive scenario walkthrough you can describe Injection A as the assumed incident; use the local excess drill when asked what the repo actually proves.

Fault	Mechanism	How to prove or diagnose it
A. Grain defect	A transaction-to-settlement join expands a payment to multiple rows before aggregation.	Compare row count and distinct payment IDs, inspect repeated keys and reconcile the numerator and denominator at payment grain.
B. Date defect	V2 filters ingestion_ts instead of transaction_ts.	Pick a late event and compare the transaction day with its ingestion day. Explain the daily KPI shift.
C. Configuration defect	V2 connects to settlement_db_v2 instead of the approved production source.	Check deployed database path, volume and dataset fingerprints, then compare counts and reporting period.
D. API calculation defect	V2 divides settled_amount by transaction_count instead of successful payment value.	Recompute from amounts and compare units. Amount divided by count is currency per payment, not a percentage.

Worked join example

A ₹1,000 payment has two settlement rows, ₹600 and ₹400. If you join first, two joined rows each carry the same ₹1,000 payment value. Summing that column returns ₹2,000. Summing the raw settlements still returns ₹1,000 in this example. A one-to-many join can distort either side depending on which measures are carried forward and how they are aggregated. If a query repeats a payment-level credited total of ₹1,000 twice while keeping a distinct ₹1,000 denominator, it reports 200%. This illustrates the defect mechanism, not a measured project result.

A corrected query aggregates completed settlements by transaction ID, joins once to successful payments, and computes capped credit and excess separately. SUM(DISTINCT amount) is unsafe because two different payments can have the same amount. The supported fix is correct grain. The app's existing SQLite constraint and Pydantic rate bound prevent ordinary reporting from accepting a rate above 100%. To reproduce HTTP 200 plus an inflated rate in a separate faulty build, isolate the demo and verify how its response model exposes the bad rate. Never weaken the normal release guardrail.

Five Whys for the assumed Injection A

1. Why was the reported settlement rate too high? The reported numerator exceeded the eligible payment value.

2. Why was the numerator too high? Payment-level values repeated after a one-to-many join.

3. Why were values repeated? The release aggregated after expanding the payment grain.

4. Why could the defect reach traffic? The gate did not catch the changed SQL path with the relevant split-settlement case.

5. Why did that gate miss it? Investigate release test coverage and enforce independent reconciliation for the exact deployed artifact. Treat this last link as a hypothesis until evidence confirms it.

The observed drill has a different causal chain. A controlled SQL UPDATE raised actual settlement in three Gold rows, credit and rate stayed capped, and a positive excess field exposed the simulated discrepancy.

16 Permanent controls and operations

Control	Implementation or next step
Grain and constraints	Implemented: one Gold row per successful payment, unique fact IDs, FK relationships and capped credit.
Split-settlement regression	Implemented: verify one payment with multiple settlements and confirm totals, cap and replay.
Independent reconciliation	Implemented as a local readiness check against Silver, with smoke assertions on totals and rate.
Release KPI gate	Compare same-scope Green rate, gap and excess before and after promotion. Automate comparison to approved baseline as follow-up.
Anomaly monitoring	Alert when excess unexpectedly rises, rate drifts or ingestion grows stale. Permanent automated alerting is follow-up.
SQL and model review	Require reviewers to identify the expected grain, join cardinality and KPI formula when Gold logic changes.
Recovery drill	Keep prior image and trusted data available. Retest route, API and business figures after rollback.

What to monitor

Batch duration, successful batch timestamp, rejection and quarantine counts, orphan retries, duplicate warnings, exception counts and excess trends. A health check measures responsiveness and schema access. A smoke check verifies actual endpoints and data shape. A business gate reconciles the data against the intended scope and baseline. A production readiness decision needs evidence from all three.

Database recovery versus traffic rollback

Traffic rollback changes Nginx routing; Green's volume remains as it was until cleanup. Database recovery restores a consistent backup, verifies affected data and replays approved newer batches if needed. Keep raw data and backups restricted. For actual multi-host production, revisit persistence, concurrency, availability, backups and identity rather than assuming a single-host Docker demo covers them.

17 Leadership presentation and peer review

Slide	Speaking point	Example or proof
1 Business problem	Operations needs to explain the settlement gap.	₹48 crore minus ₹45.8 crore leaves ₹2.2 crore in the case.
2 Architecture	Show Source, Bronze, Silver, Gold, API and Dashboard, then routing.	Trace a ₹1,000 split payment; show port 8080 through Nginx.
3 Production readiness	Walk tests, security, Docker, GitLab CI, Jenkins and Blue-green.	Six tests, zero critical, Jenkins #2 and #3. Distinguish GitLab design from measured run.
4 Production incident	Explain symptom, detection, impact and evidence.	Brief: 103.7%. Actual: ₹5.60 lakh excess at 94.70% with HTTP 200.
5 Decision	Restore trusted reporting while diagnosing Green.	Blue remained healthy with validated KPIs.
6 Recovery	V2, incident, detection, rollback, V1, validation.	Same URL, blue-v1, 94.70% and zero excess.
7 Prevention	Connect grain controls, tests and release checks to the cause.	Test split settlements; reconcile the rate and excess before and after release.
8 Business outcome	Explain visibility, investigation, trust, deployment and recovery.	From top merchant gaps to payments; recover via the same URL.

Aim for 8 to 10 minutes. Spend more time on the worked example and the incident evidence than on tool names. Lead with the problem, show how the platform computes a reliable answer, and close with how operations uses it. If asked whether the bank paid out the gap, say the platform identifies and tracks it; settlement execution is outside scope.

Peer review

Score another engineer or team on a 1-5 scale across technical implementation, data modelling, testing, security, CI/CD, incident response, communication, business understanding and team collaboration. Use the rubric from the Day 2 brief in the appendix. Support scores with evidence. Each reviewer answers what was technically strong, the biggest risk, whether the decision was supported by evidence, whether rollback was adequate, and one improvement.

18 Panel questions and answers

Question	Answer
What exactly does the dashboard fix?	It connects payment and settlement records and makes gaps traceable. Operations uses that evidence to investigate and resolve cases.
Why is the displayed gap not ₹2.2 crore?	The ₹2.2 crore belongs to the business scenario. The observed sample scope gave ₹72.46 lakh.
Is a split settlement a duplicate?	Two unique settlements can both be legitimate. An identical replay or repeated join row can cause duplicate counting.
Why include Bronze when Gold has the answer?	Bronze lets us audit and diagnose the source of a reported number or rejected row.
Why separate facts in Silver?	Payments, settlements and events have different grains. Keeping those grains avoids accidental multiplication.
Are there three databases?	One SQLite file per environment or container volume, with tables representing the layers.
What happens when an old payment settles tomorrow?	A new batch can link the settlement to the earlier payment and refresh that payment's Gold row and earlier daily KPI.
Why can the merchant table show more rows than the risk count?	The table uses OR. The risk count requires both thresholds to be breached.
Why are 401 and 422 different?	401 is an authorization problem. 422 is a request schema or parameter validation problem.
How do temporary tests get a token?	The fixture sets the expected value and sends it via the bearer header.
Why does HTTP 200 miss a bad KPI?	A valid HTTP response can carry incorrect business numbers. Compare against rules and a trusted baseline.
What is the difference between Docker and Jenkins?	Jenkins runs the build workflow. Docker builds the image and runs containers.
Why maintain GitLab CI and Jenkins?	GitLab CI is the primary design. Jenkins supports a separate build path and legacy workflows.
What is the difference between an image and a container ID?	An image identifies packaged content. A container ID identifies an instance created from an image.
Is the database baked into the image?	No. The app reads /data/settlement.db from its attached volume.
What does Nginx do?	It accepts traffic through the host's port 8080 and forwards to the active container's internal port 8000.
Why is Green on port 8083?	A direct preview can be tested before Nginx moves the shared route to it.
Does Green always have bad data?	No. Clean Green matched Blue. The fault was injected only for the drill and then cleaned up.
Why didn't the actual rate go above 100%?	We modified actual settlement on already fully settled payments. Credited stayed capped and the excess field changed.

Question	Answer
Was 103.7% actually observed?	No. It is the brief's incident example. The recorded drill measured excess at a steady rate.
Did we inject the faulty SQL join?	No. The recorded drill updated Green's Gold values. Injection A is explained as the brief scenario.
Does rollback fix Green's database?	Routing back to Blue restores what users see. Green's data requires separate restoration.
What would you improve first?	Automate same-scope rate, gap and excess reconciliation through the shared URL, with a release stop on unexpected changes.
What could still block enterprise rollout?	Outstanding high findings, verified GitLab production execution, hardened deployment and operational monitoring.

Appendix A / Requirements as supplied

The original briefs are included as extracted text. Layout and diagram spacing are adapted for this book. Read their scenarios and targets together with the execution evidence in the preceding chapters.

Day 1 brief

Bank of New York — Real-Time Merchant Settlement Intelligence Platform
Sprint Objective
The Data Engineers must take an unseen banking problem and build a working solution from specification → data model → API → pipeline → front-end → testing → implementation.
1. Business Scenario
Bank of New York provides payment acquiring services to merchants across India.
The Payments Operations team currently receives settlement information from multiple sources:
Payment Gateway
|
+------ Transaction System
|
+------ Settlement Processor
|
+------ Merchant Risk System
|
+------ Finance System
Operations currently uses Excel reports generated once every night.
The Head of Payments has identified a problem:
"Yesterday's transaction volume was ₹48 Cr, but only ₹45.8 Cr was settled. Operations cannot determine whether the difference is caused by failed payments, pending settlements, processing delays, merchant risk controls or data-quality problems."
The bank wants an engineering solution that allows operations users to:
View payment and settlement performance.
Identify settlement gaps.
investigate delayed settlements.
identify merchants with abnormal settlement behaviour.
expose the analytics through an API.
consume the API through a simple web dashboard.
process new data incrementally.
automatically test the implementation.
2. The Unseen Data Sources
The data engineers receives four CSV files.
transactions.csv
Column
Description
transaction_id
Payment transaction
merchant_id
Merchant
customer_id
Customer
transaction_ts
Transaction timestamp
amount
Transaction amount
currency
Transaction currency
status
SUCCESS / FAILED / REVERSED
payment_channel
POS / ONLINE / QR

Expected grain:
One row = one payment transaction attempt.
settlements.csv
Column
Description
settlement_id
Settlement identifier
transaction_id
Payment transaction
settlement_ts
Settlement timestamp
settlement_amount
Amount settled
settlement_status
SETTLED / PENDING / FAILED
settlement_batch
Settlement batch
Important:
One transaction can have more than one settlement record.
merchant.csv
Column
Description
merchant_id
Merchant
merchant_name
Merchant
merchant_category
Merchant category
country
Country
risk_level
LOW / MEDIUM / HIGH
effective_from
Risk effective date
effective_to
Risk expiry date
The risk level is historical.
payment_events.csv
Column
Description
event_id
Event ID
transaction_id
Transaction
event_type
CREATED / AUTHORIZED / SETTLED / FAILED
event_ts
Event occurrence time
ingestion_ts
Time received by platform
processing_ms
Processing time
Some events arrive late.
3. Hidden Data Problems
The data engineers is not told the following explicitly.
They need to discover them during implementation/testing.
Issue 1 — One-to-many settlement
Example:
Transaction T1001
|
+-- Settlement S5001 = ₹8,000

+-- Settlement S5002 = ₹2,000
A direct transaction → settlement join can produce incorrect transaction-level calculations.
Issue 2 — Late-arriving events
Example:
event_ts 10:02:15
ingestion_ts 10:08:41
The data engineers must distinguish:
business time from processing/ingestion time.
Issue 3 — Out-of-order events
Example:
AUTHORIZED 10:01
SETTLED 10:03
CREATED 10:00
The physical arrival order does not necessarily represent business event order.
Issue 4 — Merchant risk changes
Example:
M100
LOW Jan–Mar
HIGH Apr–Jun
MEDIUM Jul onwards
A transaction in February must use the February risk classification.
Issue 5 — Data-quality problems
The dataset contains:
missing merchant IDs
duplicate event IDs
negative settlement amounts
transactions without settlement
settlements without a matching transaction
invalid currency values
events arriving after the expected SLA
The data engineers must decide which are:
rejectable records
quarantine records
warning conditions
business exceptions
4. Business Requirements
The Head of Payments wants a dashboard containing:
KPI 1 — Transaction Volume
Total successful payment value.
KPI 2 — Settlement Rate
Settled transaction amount
---------- × 100
Successful transaction amount
KPI 3 — Settlement Gap
Successful transaction amount
-
Settled amount
KPI 4 — Settlement SLA
Percentage of settlements completed within 30 minutes of the successful transaction.
KPI 5 — Merchant Risk
Number of merchants with:
Settlement Rate < 95%
and
Settlement SLA < 90%
5. Engineering Requirement
The bank does not want a single Python script.
The solution must have these layers:
SOURCE FILES
|
v
INGESTION

|
v
BRONZE
|
v
SILVER
|
v
GOLD
|
v
API
|
v
FRONT-END
The implementation should support incremental processing rather than reprocessing every record unnecessarily.
6. Sprint Tasks
TASK 1 — Specification Package
Before writing implementation code, the data engineers must produce a mini specification.
1. Business Specification
Document:
problem statement
objective
users
KPIs
scope
assumptions
business rules
acceptance criteria
2. Technical Specification
Document:
data sources
fields
processing
storage
API
front-end
validation
monitoring
security
3. Gherkin
Write at least 5 acceptance scenarios.
Example:
Scenario: Identify an unsettled successful transaction
Given a successful payment exists
And no successful settlement exists for the transaction
When the settlement pipeline runs
Then the transaction should be classified as UNSETTLED
And it should appear in the settlement exception report
TASK 2 — Data Model + Data Pipeline
Build the physical model.
A reasonable target may contain:
DIM_DATE
DIM_MERCHANT
DIM_CUSTOMER
FACT_TRANSACTION
FACT_SETTLEMENT
FACT_PAYMENT_EVENT
But the data engineers must justify the model.
Mandatory
The data engineers must explicitly document:

Grain of every fact table
For example:
FACT_TRANSACTION
One row = one payment transaction attempt
FACT_SETTLEMENT
One row = one settlement event
FACT_PAYMENT_EVENT
One row = one payment lifecycle event
Implement
DDL
primary keys
foreign keys
constraints
indexes/partitioning where appropriate
staging tables
Bronze/Silver/Gold processing
Pipeline must handle
Raw
↓
Validation
↓
Clean
↓
Business transformation
↓
Gold
Invalid records should not silently disappear.
TASK 3 — API + Front-End + Full Test Suite
Part A — API
Create:
GET /api/v1/settlement-summary
Parameters:
start_date
end_date
merchant_id (optional)
Example:
GET /api/v1/settlement-summary?start_date=2026-09-01&end_date=2026-09-07
Response:
{
"transaction_count": 15240,
"transaction_amount": 482500000,
"settled_amount": 458700000,
"settlement_rate": 95.07,
"settlement_gap": 23800000,
"sla_rate": 91.4
}
Use:
FastAPI
Pydantic
OpenAPI
Part B — Merchant Exception API
Create:
GET /api/v1/merchant-exceptions
Return merchants where:
settlement_rate < 95%
OR
settlement_sla < 90%
Example:
{
"merchant_id": "M1008",
"merchant_name": "ABC Retail",

"settlement_rate": 88.4,
"sla_rate": 72.5,
"risk_level": "HIGH"
}
Part C — Front-End
Create a simple HTML/JavaScript dashboard.
It must show:
KPI cards
Transactions
₹ Transaction Volume
Settlement Rate
Settlement Gap
SLA Rate
Chart 1
Daily Transaction Amount
vs
Daily Settlement Amount
Chart 2
Top 10 merchants with settlement gaps
Table
Merchant
Risk
Settlement Rate
SLA
Settlement Gap
The front-end must consume the API rather than directly reading CSV files.
TASK 4 — Testing, Security, Deployment & Engineering Review
The data engineers must demonstrate that the solution is not just functional but engineered properly.
A. TDD / Unit Testing
Implement tests for:
successful transaction
failed transaction
duplicate event
missing merchant
unmatched settlement
negative settlement
late event
settlement calculation
B. Data Model Tests
Include tests for:
Grain
SELECT transaction_id, COUNT(*)
FROM fact_transaction
GROUP BY transaction_id
HAVING COUNT(*) > 1;
Expected:
0 rows
Referential integrity
Transactions should not reference nonexistent merchants.
Settlement reconciliation
The candidate should be able to identify:
Successful transactions
+
Settlements
+
Unsettled transactions
and reconcile the totals.
C. API Contract Tests
Test:
200 – valid request
400 – invalid date

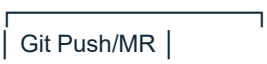
404 – unknown merchant
422 – invalid parameter
Also test that:
settlement_rate >= 0
settlement_rate <= 100
D. Security
The data engineers must identify and prevent issues such as:
SQL injection
unrestricted API access
secrets inside source code
PII leakage
excessive logging
missing authorization
For example, this should not be accepted:
query = f"""
SELECT *
FROM transactions
WHERE merchant_id = '{merchant_id}'
"""
The data engineers should use parameterized queries.
E. Deployment
Provide a deployment design:
GitLab
|
v
CI Pipeline
|
+-- Tests
+-- SAST
+-- Dependency Scan
+-- Docker Build
|
v
DEV
|
v
TEST
|
v
PROD
The candidate should identify:
environment configuration
secrets
health check
smoke test
rollback mechanism

Day 2 brief

Bank of New York — Production Readiness, Blue-Green Cutover & Incident Recovery Sprint
Continuation of the previous Bank of New York case study: Real-Time Merchant Settlement Intelligence Platform
1. Sprint Objective
The previous sprint produced:
Source Files
↓
Bronze
↓
Silver
↓
Gold
↓

FastAPI
↓
HTML/JavaScript Dashboard
The data engineers now needs to prove that this solution is production-ready.
The day's challenge has three phases:
Morning
Make the existing Bank of New York solution production-ready, ensure the complete test suite is green, pass security gates, and deploy it through GitLab CI with Jenkins as the secondary build path.
Midday
Now deliberately breaks something in production.
The data engineers must:
Detect
↓
Investigate
↓
Assess impact
↓
Decide
↓
Rollback
↓
Verify
↓
Produce evidence
Afternoon
Present the production solution and incident response to leadership, followed by a structured 360° peer review.
2. Starting Point
The data engineers starts the day with the Bank of New York Settlement Intelligence Platform from the previous case.
The existing repository contains:
straive-settlement-platform/
│
├── docs/
├── data/
├── sql/
├── pipeline/
├── api/
├── frontend/
├── tests/
├── Dockerfile
├── requirements.txt
└── .gitlab-ci.yml
The solution already supports:
payment transaction processing
settlement processing
merchant risk
Bronze/Silver/Gold
settlement KPIs
FastAPI
Pydantic
HTML/JavaScript dashboard
automated tests
The data engineers's responsibility today is not to rebuild the application.
They must make it deployable, observable, recoverable and demonstrably production-ready.
3. Production Requirements
Bank of New York defines the following production requirements.
Requirement
Target
Test suite
100% required tests passing
Critical security findings
0

API health
HTTP 200
API response
<500 ms under normal load
Deployment
GitLab CI
Secondary build path
Jenkins
Deployment strategy
Blue-Green
Rollback
Required
Production secrets
No hard-coded secrets
Production smoke test
Mandatory
Data reconciliation
Mandatory
Evidence
Mandatory
4. Morning Sprint — Production Deployment
Task 1 — Production Readiness & Test Gate
The data engineers must first prove that the existing application is ready to deploy.
Run:
Unit Tests
↓
Data Model Tests
↓
Pipeline Tests
↓
API Tests
↓
Business Rule Tests
↓
Security Tests
The candidate must demonstrate:
Functional tests
settlement calculation
settlement gap
SLA calculation
merchant exception logic
late-arriving event handling
Data tests
fact-table grain
duplicate detection
referential integrity
transaction/settlement reconciliation
invalid data handling
API tests
200 valid request
400 invalid request
404 invalid merchant
422 invalid parameter
Mandatory negative test
The data engineers must prove that the system does not calculate an incorrect settlement rate when multiple settlement records exist for one transaction.
5. Task 2 — Security Gate
The data engineers must integrate or execute the security checks required by the previous module.
Pipeline:





↓
Unit Tests
↓
SAST
↓
Secret Scan
↓
SCA
↓
Docker Build
↓
Container Scan
↓
Smoke Test
Required checks
SAST
Identify:
insecure SQL
unsafe input handling
insecure code patterns
Secret scanning
The repository must contain:
NO API keys
NO passwords
NO cloud credentials
NO database passwords
SCA
Check application dependencies.
The data engineers must identify:
vulnerable dependency
severity
whether it blocks production
remediation approach
Container scan
Scan the Docker image.
6. Task 3 — GitLab CI Production Pipeline
The data engineers must complete the GitLab pipeline.
Required flow:
validate
↓
test
↓
security
↓
build
↓
package
↓
deploy_dev
↓
smoke_test
↓
deploy_prod
Production deployment must require a controlled approval.
Example:
deploy_prod:
stage: deploy
when: manual
The data engineers must demonstrate appropriate use of:
stages

jobs
artifacts
cache
rules
needs
protected variables
environments
manual approval

7. Task 4 — Jenkins Secondary Build Path

Bank of New York's engineering organisation maintains Jenkins for legacy enterprise workloads.
The data engineers must create a Jenkinsfile that can independently:

Checkout
↓
Install Dependencies
↓
Run Tests
↓
Security Checks
↓
Build Docker Image
↓
Publish Artifact

Jenkins is not the primary production deployment path.
The architecture is:

GitLab Repository
|
┌──────────────┴──────────────┐
↓ ↓
GitLab CI Jenkins
Primary Path Secondary Path
| |
└──────────────┬──────────────┘
↓
Artifact
|
↓
Production

The data engineers must explain:
Why would an enterprise maintain Jenkins alongside GitLab CI?

8. Task 5 — Blue-Green Production Deployment

Bank of New York currently has:
BLUE
Version 1
The data engineers deploys:
GREEN
Version 2
Architecture:

Load Balancer
|
┌──────┴──────┐
↓ ↓
BLUE GREEN
V1 V2

Initially:
100% traffic → BLUE
After successful validation:
100% traffic → GREEN
The data engineers must perform:
Pre-cutover checks
application health
API health

database connectivity
pipeline health
test suite
smoke test
KPI reconciliation
Smoke test
GET /health
GET /api/v1/settlement-summary
GET /api/v1/merchant-exceptions
Verify:
HTTP 200
Valid JSON
Correct KPI values
Response time acceptable
Only after successful validation:
BLUE → GREEN
9. Production Evidence Package
Before moving to the incident phase, the data engineers must capture:
01_test_results
02_security_results
03_gitlab_pipeline
04_jenkins_pipeline
05_docker_image
06_deployment_version
07_blue_green_status
08_smoke_test_results
09_kpi_reconciliation
The important point is:
"It worked" is not sufficient. The data engineers must prove it worked.
10. MIDDAY — Injected Production Incident
Now deliberately breaks production.
Incident scenario
After the blue-green cutover, the operations dashboard starts showing:
Settlement Rate: 103.7%
Previously:
Settlement Rate: 95.2%
At the same time, operations reports that:
"Several merchants are showing settlement amounts greater than their successful transaction amounts."
The API is still returning:
HTTP 200
Therefore, a simple health check does not identify the problem.
11. Incident Task — Detect, Decide, Recover
The data engineers must treat this as a real production incident.
Step 1 — Detect
Identify the abnormality.
They should compare:
Previous production
vs
Current production
Investigate:
API response
application logs
deployment version
KPI results
database/query output
pipeline artifacts
Step 2 — Determine Business Impact
The data engineers must answer:
Which KPI is affected?
Settlement Rate
Settlement Gap

Merchant Exceptions
Is the issue:
application failure?
data problem?
deployment problem?
query problem?
configuration problem?
pipeline problem?
12. Hidden SQL Injection
Injection A — Grain defect
A new query joins:
FACT_TRANSACTION
JOIN
FACT_SETTLEMENT
before aggregation.
Result:
Settlement amount inflated
Injection B — Date defect
Version 2 accidentally filters using:
ingestion_ts
instead of:
transaction_ts
Result:
daily KPI values shift unexpectedly
Injection C — Configuration defect
Version 2 points to:
settlement_db_v2
instead of the approved production data source.
Injection D — API calculation defect
The API calculates:
settled_amount / transaction_count
instead of:
settled_amount / successful_transaction_amount
13. Incident Decision
The data engineers must decide:
Do we rollback or continue troubleshooting Version 2 in production?
They must provide evidence supporting the decision.
A strong incident response should contain:
Observed symptom
↓
Evidence collected
↓
Business impact
↓
Technical diagnosis
↓
Risk assessment
↓
Decision
↓
Rollback / Fix
↓
Validation
14. Rollback
If rollback is selected:
GREEN V2
↓
Traffic removed
↓
BLUE V1
↓

Smoke tests
↓
KPI validation
↓
Production restored
The data engineers must demonstrate:
Before rollback
Settlement Rate = 103.7%
After rollback
Settlement Rate ≈ expected baseline
The exact numbers should come from the trainer's dataset rather than being assumed.
15. Rollback Evidence
The data engineers must produce:
Incident timeline
10:00 – V2 deployment
10:08 – Blue-green cutover
10:15 – KPI anomaly detected
10:22 – Investigation started
10:35 – Root cause identified
10:40 – Rollback initiated
10:45 – V1 restored
10:50 – Smoke tests passed
10:55 – Business KPI reconciled
The timestamps are examples; data engineerss must use their actual evidence.
Incident report
Incident ID:
Date:
Application:
Production Version:
What happened?
Business impact:
Detection method:
Technical root cause:
Evidence:
Decision:
Rollback performed:
Validation after rollback:
Preventive action:
Owner:
Due date:
16. Root Cause Analysis
The data engineers must perform a lightweight 5 Whys.
Example structure:
Why was settlement rate >100%?
↓
Settlement amount was inflated.
↓
Why?
↓
Multiple settlement rows were joined to transactions.
↓
Why?
↓
Version 2 changed the aggregation logic.
↓
Why was this not detected?
↓
Regression test did not cover multiple settlements.
↓
Why?
↓

The test dataset contained only one settlement per transaction.
The data engineers must derive the actual chain from the injected incident.
17. Preventive Engineering Action
The data engineers must identify at least 3 permanent controls.
Examples:
Control 1
Add a regression test for:
one transaction → multiple settlements
Control 2
Add a reconciliation gate:
Settlement amount
≤
Successful transaction amount
where that relationship is valid for the defined business rules.
Control 3
Add production KPI anomaly monitoring.
Control 4
Require SQL/data-model review for changes affecting Gold-layer metrics.
Control 5
Add automated blue-green smoke tests against business KPIs rather than only /health.
18. AFTERNOON — Leadership Presentation
Task 6 — Executive Presentation
Each data engineers gets 8–10 minutes.
The presentation must answer:
Slide 1 — Business Problem
What problem does Bank of New York's Settlement Intelligence Platform solve?
Slide 2 — Architecture
Show:
Source
↓
Bronze
↓
Silver
↓
Gold
↓
API
↓
Dashboard
plus deployment architecture.
Slide 3 — Production Readiness
Show:
Tests □
Security □
Docker □
GitLab CI □
Jenkins □
Blue-Green □
Slide 4 — Production Incident
Explain:
what happened
how it was detected
impact
evidence
Slide 5 — Decision
Explain:
Why did you rollback or continue with the deployment?
Slide 6 — Recovery
Show:
V2
↓

Incident
↓
Detection
↓
Rollback
↓
V1
↓
Validation
Slide 7 — Prevention
Show the controls added to prevent recurrence.
Slide 8 — Business Outcome
Explain how the solution improves:
settlement visibility
operational investigation
data trust
deployment safety
recovery capability
19. 360° Peer Review
Task 7 — Peer Review
Each data engineers reviews another data engineers/team.
Use a 1–5 scale.
Area
1–2
3
4–5
Technical implementation
Major gaps
Functional
Strong
Data modelling
Significant issues
Adequate
Well justified
Testing
Limited
Reasonable
Comprehensive
Security
Major gaps
Basic controls
Strong controls
CI/CD
Incomplete
Working
Well engineered
Incident response
Unclear
Adequate
Evidence-driven
Communication
Difficult to follow
Clear
Structured
Business understanding
Weak
Good
Strong
Team collaboration
Limited
Good

Strong
Peer-review questions
Each reviewer must answer:
What was technically strong?
What was the biggest engineering risk?
Was the production decision supported by evidence?
Was the rollback strategy adequate?
What one improvement would you recommend?

Appendix B / Repository source and runbooks

Listings preserve the repository source for review. Line wrapping is for reading only. Run the checked-in files when reproducing the project.

.dockerignore

```
var
.venv
.git
.env
**/__pycache__
.pytest_cache
reports
evidence
```

.gitattributes

```
*.sh text eol=lf
```

.gitignore

```
__pycache__/
.pytest_cache/
.venv/
var/
.env
evidence/
.cache/
reports/
```

.gitlab-ci.yml

```
stages: [validate, test, security, build, package, deploy_dev, smoke_test, deploy_prod]

# The Docker socket is available on the dedicated project runner.
# Only trusted commits on the protected default branch can use that runner.
workflow:
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH && $CI_COMMIT_REF_PROTECTED == "true"'

default:
  tags: [capstone-docker]

.python:
  image: python:3.11-slim
  variables:
    PIP_CACHE_DIR: "$CI_PROJECT_DIR/.cache/pip"
    PIP_DISABLE_PIP_VERSION_CHECK: "1"
  cache:
    key:
      files: [requirements.txt, requirements-dev.txt]
    paths: [.cache/pip/]

.docker:
  image: docker:29-cli
  before_script:
    - docker info --format '{{.ServerVersion}}'
    - printf '%s' "$CI_REGISTRY_PASSWORD" | docker login --username "$CI_REGISTRY_USER" --password-stdin
     "$CI_REGISTRY"

validate:
  stage: validate
  extends: .python
  script:
    - python -m compileall -q settlement tests scripts
    - sh -n scripts/deploy_ci.sh
    - test -s Dockerfile

test:
  stage: test
  extends: .python
  needs: [validate]
  script:
    - mkdir -p reports
    - python -m pip install -r requirements-dev.txt
    - python -m pytest -v --junitxml=reports/pytest.xml
  artifacts:
    when: always
    reports:
      junit: reports/pytest.xml
    paths: [reports/pytest.xml]

sast:
  stage: security
  extends: .python
  needs:
    - job: test
      artifacts: false
  script:
    - mkdir -p reports
    - python -m pip install bandit==1.9.4
    - python -m bandit -r settlement -f json -o reports/bandit.json
  artifacts:
    when: always
    paths: [reports/bandit.json]

secret_scan:
  stage: security
  image:
    name: ghcr.io/gitleaks/gitleaks:v8.30.1
    entrypoint: [""]
  variables:
```

```
    GIT_DEPTH: "0"
  needs:
    - job: test
      artifacts: false
  script:
    - mkdir -p reports
    - gitleaks git --redact=100 --no-banner --report-format json --report-path reports/gitleaks.json .
  artifacts:
    when: always
    paths: [reports/gitleaks.json]

dependency_scan:
  stage: security
  extends: .python
  needs:
    - job: test
      artifacts: false
  script:
    - mkdir -p reports
    - python -m pip install pip-audit==2.10.1
    - python -m pip_audit -r requirements.txt -f json -o reports/dependencies.json
  artifacts:
    when: always
    paths: [reports/dependencies.json]

build:
  stage: build
  extends: .docker
  needs:
    - job: sast
      artifacts: false
    - job: secret_scan
      artifacts: false
    - job: dependency_scan
      artifacts: false
  script:
    - |
      set -eu
      candidate="$CI_REGISTRY_IMAGE:candidate-$CI_PIPELINE_ID-$CI_JOB_ID"
      docker build --pull --label "org.opencontainers.image.revision=$CI_COMMIT_SHA" -t "$candidate" .
      docker push "$candidate"
      digest="$(docker image inspect --format '{{index .RepoDigests 0}}' "$candidate")"
      case "$digest" in "$CI_REGISTRY_IMAGE"@sha256:*) ;; *) exit 1 ;; esac
      printf 'CANDIDATE_IMAGE=%s\n' "$digest" > candidate.env
  artifacts:
    reports:
      dotenv: candidate.env

package:
  stage: package
  extends: .docker
  needs: [build]
  script:
    - |
      set -eu
      mkdir -p reports
      export TRIVY_USERNAME="$CI_REGISTRY_USER" TRIVY_PASSWORD="$CI_REGISTRY_PASSWORD"
      docker run --rm -e TRIVY_USERNAME -e TRIVY_PASSWORD
    aquasec/trivy@sha256:8f6bb947dbc528902e11f2f5be52f7b519f6a39d5d148f41166f2040499768fb \
        image --image-src remote --scanners vuln --format json --no-progress "$CANDIDATE_IMAGE" >
    reports/trivy-image.json
      critical="$(jq '[.Results[]?.Vulnerabilities[]? | select(.Severity == "CRITICAL")] | length'
    reports/trivy-image.json)"
      echo "Critical findings $critical"
      test "$critical" -eq 0
      printf 'RELEASE_IMAGE=%s\n' "$CANDIDATE_IMAGE" > release.env
  before_script:
    - apk add --no-cache jq
  artifacts:
    when: always
    paths: [reports/trivy-image.json]
    reports:
      dotenv: release.env
```

```
deploy_dev:
  stage: deploy_dev
  extends: .docker
  needs: [package]
  resource_group: capstone-dev
  environment:
    name: development
    deployment_tier: development
  script:
    - sh scripts/deploy_ci.sh development

smoke_test:
  stage: smoke_test
  extends: .docker
  needs:
    - job: deploy_dev
      artifacts: false
    - job: package
      artifacts: true
  resource_group: capstone-dev
  environment:
    name: development
    action: verify
  script:
    - mkdir -p reports
    - test "$(docker inspect --format '{{.Config.Image}}' capstone-development)" = "$RELEASE_IMAGE"
    - docker exec -i capstone-development python - < scripts/smoke_ci.py > reports/dev-smoke.json
    - cat reports/dev-smoke.json
  artifacts:
    when: always
    paths: [reports/dev-smoke.json]

deploy_prod:
  stage: deploy_prod
  extends: .docker
  needs:
    - job: smoke_test
      artifacts: false
    - job: package
      artifacts: true
  rules:
    - if: '$CI_COMMIT_BRANCH == $CI_DEFAULT_BRANCH && $CI_COMMIT_REF_PROTECTED == "true"'
      when: manual
      allow_failure: false
  manual_confirmation: "Deploy the verified image to the production environment?"
  resource_group: capstone-prod
  environment:
    name: production
    deployment_tier: production
  script:
    - sh scripts/deploy_ci.sh production
```

Dockerfile

```
FROM python:3.11-slim
ENV PYTHONDONTWRITEBYTECODE=1 PYTHONUNBUFFERED=1 SETTLEMENT_DB=/data/settlement.db
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt && useradd --create-home --uid 10001 settlement &&
    mkdir /data && chown settlement:settlement /data
COPY settlement ./settlement
USER settlement
EXPOSE 8000
HEALTHCHECK --interval=30s --timeout=3s CMD python -c "import urllib.request;
    urllib.request.urlopen('http://localhost:8000/health', timeout=2)"
CMD ["python", "-m", "uvicorn", "settlement.api:app", "--host", "0.0.0.0", "--port", "8000", "--no-
    access-log"]
```

Jenkinsfile

```
pipeline {
    agent { label 'capstone-windows' }

    options {
        skipDefaultCheckout()
        disableConcurrentBuilds()
        timeout(time: 30, unit: 'MINUTES')
        buildDiscarder(logRotator(numToKeepStr: '10', artifactNumToKeepStr: '3'))
    }

    environment {
        REPORT_DIR = "reports/${BUILD_NUMBER}"
        APP_IMAGE = "capstone-settlement:jenkins-${BUILD_NUMBER}"
        TRIVY_IMAGE =
    'aquasec/trivy@sha256:8f6bb947dbc528902e11f2f5be52f7b519f6a39d5d148f41166f2040499768fb'
    }

    stages {
        stage('Checkout') {
            steps {
                checkout scm
            }
        }
        stage('Install dependencies') {
            steps {
                powershell '''
                    New-Item -ItemType Directory -Force $env:REPORT_DIR | Out-Null
                    python -m venv .venv
                    if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
                    .venv/Scripts/python.exe -m pip install -r requirements-dev.txt bandit==1.9.4 pip-
    audit==2.10.1
                    exit $LASTEXITCODE
                '''
            }
        }
        stage('Test') {
            steps {
                powershell '''
                    .venv/Scripts/python.exe -m pytest -v --junitxml="$env:REPORT_DIR/pytest.xml"
                    exit $LASTEXITCODE
                '''
            }
        }
        stage('Security checks') {
            steps {
                powershell '''
                    $failed = $false
                    .venv/Scripts/python.exe -m bandit -r settlement -f json -o
    "$env:REPORT_DIR/bandit.json"
                    if ($LASTEXITCODE -ne 0) { $failed = $true }
                    .venv/Scripts/python.exe -m pip_audit -r requirements.txt -f json -o
    "$env:REPORT_DIR/dependencies.json"
                    if ($LASTEXITCODE -ne 0) { $failed = $true }

                    $repo = (Get-Location).Path
                    $reports = (Resolve-Path $env:REPORT_DIR).Path
                    docker run --rm --mount "type=bind,source=$repo,target=/src,readonly" --mount
    "type=bind,source=$reports,target=/out" -e GIT_CONFIG_COUNT=1 -e GIT_CONFIG_KEY_0=safe.directory -e
    GIT_CONFIG_VALUE_0=/src ghcr.io/gitleaks/gitleaks:v8.30.1 git --redact=100 --no-banner --report-
    format json --report-path /out/gitleaks.json /src
                    if ($LASTEXITCODE -ne 0 -or $failed) { exit 1 }
                '''
            }
        }
        stage('Build and scan image') {
            steps {
                powershell '''
                    $ErrorActionPreference = 'Stop'
                    docker build --pull --label "org.opencontainers.image.revision=$env:GIT_COMMIT" -t
    $env:APP_IMAGE .
```

```
                    if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
                    $imageId = docker image inspect --format '{{.Id}}' $env:APP_IMAGE
                    if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
                    $reports = (Resolve-Path $env:REPORT_DIR).Path
                    $archive = Join-Path $reports 'capstone-image.tar'
                    docker save -o $archive $env:APP_IMAGE
                    if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
                    docker run --rm --mount "type=bind,source=$archive,target=/image.tar,readonly"
    --mount "type=bind,source=$reports,target=/out" $env:TRIVY_IMAGE image --input /image.tar --scanners
     vuln --no-progress --format json --output /out/trivy-image.json --exit-code 0
                    if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }

                    $scan = Get-Content (Join-Path $reports 'trivy-image.json') -Raw | ConvertFrom-Json
                    if (!$scan.Results) { throw 'Trivy produced no scan results' }
                    $findings = @($scan.Results | ForEach-Object Vulnerabilities | Where-Object { $_ })
                    $critical = @($findings | Where-Object Severity -eq 'CRITICAL').Count
                    $sha256 = [System.Security.Cryptography.SHA256]::Create()
                    $stream = [System.IO.File]::OpenRead($archive)
                    try {
                        $archiveHash =
    [System.BitConverter]::ToString($sha256.ComputeHash($stream)).Replace('-', '')
                    } finally {
                        $stream.Dispose()
                        $sha256.Dispose()
                    }
                    [ordered]@{
                        image = $env:APP_IMAGE
                        image_id = $imageId
                        commit = $env:GIT_COMMIT
                        archive_sha256 = $archiveHash
                        critical_findings = $critical
                    } | ConvertTo-Json | Set-Content (Join-Path $reports 'image.json')
                    if ($critical -gt 0) { throw "Image has $critical critical findings" }
                '''
            }
        }
        stage('Publish artifact') {
            steps {
                archiveArtifacts artifacts: "${env.REPORT_DIR}/capstone-image.tar", fingerprint: true
            }
        }
    }

    post {
        always {
            junit testResults: "${env.REPORT_DIR}/pytest.xml", allowEmptyResults: true
            archiveArtifacts artifacts: "${env.REPORT_DIR}/*.json,${env.REPORT_DIR}/*.xml",
    allowEmptyArchive: true
        }
    }
}
```

README.md

````
# Merchant Settlement Intelligence

A full-stack capstone for Payments Operations. The ingestion command loads CSVs into Bronze, validates
    them into Silver, and refreshes affected Gold payment records. FastAPI serves the HTML/JavaScript
    dashboard from that database.

## Run locally

From this folder in PowerShell:

```powershell
python -m venv .venv
.venv/Scripts/python.exe -m pip install -r requirements-dev.txt
$data = 'C:/Users/PARTHA/OneDrive/Desktop/capstone2_datasets'
.venv/Scripts/python.exe -m settlement.ingest $data
.venv/Scripts/python.exe -m settlement.ingest "$data/incremental"
.\start.ps1
```

Open http://127.0.0.1:8000 and enter the token printed in your terminal. The initial dashboard spans all
     loaded payment dates. Use /docs for the API contract. The token is kept in memory by the dashboard.
     The default database is var/settlement.db. Set SETTLEMENT_DB to use a different location for both
    commands.

A batch folder contains all four CSVs with base names or a shared date suffix. Submit the incremental
    folder again to verify replay reports zero affected payments. A new batch can be loaded while the
    app is running. Apply the dashboard filters again to fetch updated results.

Run the isolated automated tests with `.venv/Scripts/python.exe -m pytest -q`. See [business
    specification](docs/business.md), [technical specification](docs/technical.md), [acceptance
    scenarios](tests/acceptance.feature), and [deployment design](docs/deployment.md).
````

docs/business.md

````
# Business specification

Payments Operations needs to explain differences between successful payment value and money settled. The
     Head of Payments monitors overall performance. Analysts filter by inclusive payment dates (India
    local time) and optionally merchant, review daily totals and ranked merchant gaps, and inspect
    outstanding and delayed payments. Scope covers analytics on the supplied exports, including
    incremental batches. Payment execution, exchange rate conversion, and attributing holds to specific
    bank risk controls are outside scope.

Successful payment attempts determine transaction count and INR volume. For each attempt, credit only
    completed settlement rows, capped at the original payment amount. Report excess separately as a
    warning and preserve the actual amount for reconciliation. Gap is successful value less credited
    settlement value. Settlement rate is credited value divided by successful value. A successful
    payment is UNSETTLED, PARTIAL, or SETTLED based on credited value. Complete at the first timestamp
    where cumulative settlement meets the original amount. SLA divides completions within 30 minutes
    (inclusive) by fully completed payments. Rates with no denominator are null. Partial or unsettled
    payments older than 30 minutes are overdue as of request time.

The merchant risk KPI counts merchants breaching both settlement rate <95% and SLA <90%. The exceptions
    table uses either threshold. A null rate does not breach its threshold. Each payment retains the
    merchant risk effective when it occurred. For merchant summaries display the risk on the most recent
     successful payment in the selected period. Daily charts group both values by payment date so an
    older day can change after a new batch.

Acceptance requires an initial and incremental load, replay with no double counting, reconciliation to
    the cent, historical risk, visible exceptions, the requested dashboard and API contracts, and
    automated verification.
````

docs/deployment.md

```
# Deployment and engineering review

GitLab runs unit, data model and API tests, then Bandit static analysis and pip-audit dependency
    scanning before building and pushing an image tagged by commit. The build needs a runner that
    supports Docker-in-Docker. Promote the same reviewed image digest through DEV, TEST, and PROD. Each
    environment has its own SQLite volume, input mount, access token, and network permissions.

1. DEV uses the base and incremental CSVs. Run the image with the data volume and a read-only CSV mount,
     overriding its command with python -m settlement.ingest /input. Start the app with the same data
    volume.
2. TEST verifies authentication, dashboard filters, smoke requests to /health and the summary endpoint,
    and unchanged totals after incremental replay. Verify that failed ingestion leaves the previous
    report available.
3. PROD mounts the same environment's persistent data volume into both the ingestion command and app.
    The container UID 10001 needs write access to that volume for ingestion and SQLite WAL files. Supply
     SETTLEMENT_API_TOKEN through the deployment secret manager, restrict app traffic to a TLS reverse
    proxy, and rotate tokens as needed. Use synthetic data outside production. A shared bearer token
    provides one operations role. An organization rollout can replace it with SSO for individual access
    and auditing.

Record ingestion CLI output centrally with restricted access. Alert on failed runs, stale ingestion, or
    changes in finding rates. Use /health for readiness. Archive delivered batches securely and take a
    consistent SQLite backup with the SQLite backup API before database changes. Roll back a bad
    application release by redeploying the prior image digest. For a database rollback, stop ingestion,
    restore the tested backup, restart the previous image, and replay verified batches delivered after
    the backup. Keep Bronze and its customer identifiers outside web-accessible storage. The capstone
    uses a single host and SQLite volume. Reassess storage and identity for multi-host production.
```

docs/gitlab-task3.md

```
# Day 2 Task 3 GitLab pipeline

The pipeline runs on a dedicated GitLab project runner backed by Docker Desktop's Linux engine. A push
    to the protected default branch starts validation, pytest, Bandit, Gitleaks and pip-audit. The build
     pushes a uniquely tagged candidate to the project registry. Trivy scans the candidate by digest.
    The package job releases that digest to the deployment jobs only if the scan completes with zero
    critical findings. GitLab keeps the test, scan and DEV smoke reports as job artifacts.

## Set up the project runner

1. In GitLab, protect the default branch and enable the project's Container Registry. Under **Settings →
     CI/CD → Runners**, create a project runner with tag `capstone-docker`. Mark it **Protected** and
    keep it dedicated to this project. The Docker socket gives its jobs control over Docker Desktop, so
    restrict writes to the protected branch to trusted maintainers.
2. With Docker Desktop running in Linux mode, start a runner container in PowerShell. Use the
    registration command shown by GitLab with its authentication token. Keep that token out of source
    control.

```powershell
docker volume create capstone-runner-config
docker run -d --name capstone-runner --restart unless-stopped -v capstone-runner-config:/etc/gitlab-
    runner -v /var/run/docker.sock:/var/run/docker.sock gitlab/gitlab-runner:alpine
docker exec -it capstone-runner gitlab-runner register --url https://gitlab.com --executor docker
    --docker-image docker:29-cli --docker-volumes /var/run/docker.sock:/var/run/docker.sock
```

When prompted, provide the runner authentication token from GitLab. Confirm that the runner is online in
     the project before pushing to the default branch.

3. In **Settings → CI/CD → Variables**, add a masked, protected `SETTLEMENT_API_TOKEN` scoped to
    `development`. Add a separate value for the `production` scope before using the manual job. Use
    GitLab's built-in registry variables for registry access. If protected environments are available in
     your GitLab tier, restrict who may deploy to `production`.

Prepare the DEV database

The deployment expects a preloaded SQLite database at `/data/settlement.db` in a volume called
    `capstone-development-data`. Seed it once with approved demo data. The following assumes the Day 1
    database exists locally and Docker has the Day 2 app image.

```powershell
.venv/Scripts/python.exe -c "import sqlite3;
    a=sqlite3.connect('file:var/settlement.db?mode=ro',uri=True); b=sqlite3.connect('var/ci-seed.db');
    a.backup(b); b.execute('PRAGMA journal_mode=DELETE'); b.close(); a.close()"
$seed = (Resolve-Path var/ci-seed.db).Path
docker volume create capstone-development-data
docker run --rm --user 0 --entrypoint sh --mount "type=bind,source=$seed,target=/seed.db,readonly"
    --mount "type=volume,source=capstone-development-data,target=/data" capstone-
    settlement:day2-security -ec 'test ! -e /data/settlement.db; cp /seed.db /data/settlement.db; chown
    10001 /data/settlement.db'
```

DEV is available on `http://127.0.0.1:8081` on the Docker host. Its smoke check validates health, data
    availability, the summary, and the merchant exceptions response. The manual production job expects
    its own approved data volume, `capstone-production-data`, writable by the application's UID 10001.
    It preflights a candidate before replacing the container on port 8082 and restores the previous
    container if the replacement fails its smoke check. Traffic routing for the full blue-green exercise
     belongs to Task 5.

For Task 3 evidence, capture the pipeline page, the artifact reports, and the manual production job
    waiting for approval. Click that job only when the intended production or capstone demo environment
    is ready.
```

docs/jenkins-task4.md

```
# Day 2 Jenkins build

Jenkins runs the secondary build path from the same GitLab repository. It checks out the commit,
    installs dependencies, runs pytest, scans source and dependencies, builds a Docker image and scans
    it with Trivy. A successful build publishes a downloadable Docker archive. The GitLab pipeline
    remains the primary deployment configuration.

1. In Jenkins, use a Windows agent with the label `capstone-windows`. From the agent's account, confirm
    that `git --version`, `python --version` and `docker version` work. Docker Desktop must be running
    in Linux container mode. Use a dedicated agent for this trusted project because Docker access gives
    the build control of the local engine.
2. Push the Jenkinsfile to the project. Create a **Pipeline** job in Jenkins. Set **Definition** to
    **Pipeline script from SCM**, **SCM** to **Git**, repository URL to `https://gitlab.com/straive-
    group303444/capstone_final.git`, branch to `*/main`, and **Script Path** to `Jenkinsfile`. Add
    Jenkins SCM credentials if the project requires them.
3. Click **Build Now**. The **Test Result** and build artifacts are the evidence for Task 4. Download
    the JSON scan reports, pytest XML and `capstone-image.tar` from the build. The image archive is
    published after the zero-critical gate passes.

Load a downloaded artifact into Docker Desktop with `docker load -i capstone-image.tar`. Use the image
    name and ID recorded in `image.json` to identify the build.

Jenkins can complete this build without GitLab CI. The Jenkins job still uses GitLab to fetch the
    repository. The Jenkins build record can demonstrate the secondary path, while the GitLab pipeline
    record is separate Task 3 evidence.

For a local capstone demo with a dedicated Jenkins instance, open **Manage Jenkins → Nodes → Built-In
    Node → Configure**. Set **Number of executors** to `1` and add the label `capstone-windows`. Install
     the suggested plugins during initial setup so the Pipeline, Git and JUnit steps are available.
```

docs/technical.md

```
# Technical specification

Python reads the four named CSVs with the standard csv library. Each ingestion command receives one
    folder, with four base filenames or four consistently dated filenames. SQLite stores the physical
    model and serves the single-host capstone deployment. All source timestamps are India local time.
    Merchant end dates are inclusive, and a blank end date is open. Store monetary amounts as integer
    paise.

Bronze stores row payload, file type, row number, and batch ID. A digest identifies a batch for replay
    protection. Silver has merchant history (one row per merchant and effective start), payment attempts
     (one row per transaction ID), settlement facts (one row per settlement ID), and payment events (one
     row per event ID). Constraints and indexes protect the fact grain and references. Customer ID is
    retained in the source payload only. Gold has one row per successful transaction; reporting queries
    aggregate this table by day and merchant. Rebuild Gold only for affected transactions. Merchant
    history additions may change previously unmatched rows, and orphan settlements/events are retried
    when their parent arrives.

Reject invalid fields, unsupported currencies and negative settlement amounts. Quarantine missing
    references, conflicting IDs and overlapping merchant history. Ignore identical business records and
    record duplicate events as warnings. Warn for event delivery lag beyond 30 minutes (an operational
    assumption separate from settlement SLA) and excess settlement. Order lifecycle events by event
    timestamp and ID. Store source row lineage on findings. Settlement timestamps earlier than payment
    timestamps are invalid. A pending settlement may transition to a terminal state under the same ID
    and transaction. No settlement is a business exception.

FastAPI serves settlement-summary and merchant-exceptions plus daily-trend, merchant-gaps and payment-
    exceptions under /api/v1. Dates are ISO calendar dates and inclusive. Invalid calendar dates or
    reversed bounds return 400, unknown merchant 404, missing or invalid parameters 422. The HTML
    dashboard calls same-origin JSON endpoints. Endpoints use parameterized SQL and require an
    operations bearer token set through the environment. Tokens and customer fields are excluded from
    API responses and application logs. Production requires TLS, restricted network ingress, database
    file permissions and managed secrets.

Monitor batch duration, rows loaded, rejections, quarantine, warnings and the last successful load.
    Health probes check database schema availability. Commit ingestion and affected Gold updates
    atomically. Deploy the same scanned container to DEV, TEST, and PROD with environment-specific
    secrets and database volumes.
```

requirements-dev.txt

```
-r requirements.txt
pytest>=8.3.3,<10
httpx>=0.27.2,<1
```

requirements.txt

```
fastapi==0.162.3
uvicorn==0.54.0
pydantic==2.14.3
tzdata>=2025.2,<2027
```

scripts/day2_readiness.py

```
"""Capture a local Blue V1 baseline and live HTTP latency evidence."""

import argparse
import hashlib
import json
import math
import os
import platform
import secrets
import socket
import sqlite3
import statistics
import subprocess
import sys
import time
from concurrent.futures import ThreadPoolExecutor
from contextlib import closing
from datetime import datetime, timezone
from pathlib import Path

import httpx


def reconcile(db):
    dates = db.execute(
        "SELECT min(date(ts)), max(date(ts)) FROM payment WHERE status='SUCCESS'"
    ).fetchone()
    if not dates[0]:
        raise RuntimeError("Load the capstone dataset before running this check")
    merchants = db.execute("""
        WITH running AS (
            SELECT transaction_id, ts, amount,
                   sum(amount) OVER (PARTITION BY transaction_id ORDER BY ts,id) AS cumulative
            FROM settlement WHERE status='SETTLED'
        ), totals AS (
            SELECT r.transaction_id, sum(r.amount) AS actual,
                   min(CASE WHEN r.cumulative >= p.amount THEN r.ts END) AS completed_at
            FROM running r JOIN payment p ON p.id=r.transaction_id
            GROUP BY r.transaction_id
        )
        SELECT p.merchant_id, count(*) AS n, sum(p.amount) AS amount,
               sum(min(p.amount,coalesce(t.actual,0))) AS settled,
               sum(coalesce(t.actual,0)) AS actual, count(t.completed_at) AS completed,
               sum(CASE WHEN unixepoch(t.completed_at)-unixepoch(p.ts)<=1800
                        THEN 1 ELSE 0 END) AS on_time
        FROM payment p LEFT JOIN totals t ON p.id=t.transaction_id
        WHERE p.status='SUCCESS' GROUP BY p.merchant_id
    """).fetchall()
    totals = {key: sum(row[key] for row in merchants)
              for key in ("n", "amount", "settled", "actual", "completed", "on_time")}
    expected = {
        "transaction_count": totals["n"],
        "transaction_amount": totals["amount"] / 100,
        "settled_amount": totals["settled"] / 100,
        "settlement_gap": (totals["amount"] - totals["settled"]) / 100,
        "settlement_rate": round(100 * totals["settled"] / totals["amount"], 2),
        "sla_rate": round(100 * totals["on_time"] / totals["completed"], 2)
        if totals["completed"] else None,
        "excess_settled_amount": (totals["actual"] - totals["settled"]) / 100,
        "merchant_risk_count": sum(
            row["settled"] * 100 < row["amount"] * 95
            and row["on_time"] * 100 < row["completed"] * 90
            for row in merchants
        ),
    }
    exceptions = sorted(
        row["merchant_id"] for row in merchants
        if row["settled"] * 100 < row["amount"] * 95
        or row["on_time"] * 100 < row["completed"] * 90
    )
```

```
    return dict(zip(("start_date", "end_date"), dates)), expected, exceptions


def save(folder, name, content):
    (folder / name).write_text(json.dumps(content, indent=2) + "\n", encoding="utf-8")


def measure(client, endpoints, filters):
    def request(path):
        start = time.perf_counter()
        response = client.get(path, params=filters)
        elapsed = (time.perf_counter() - start) * 1000
        return {"endpoint": path, "status": response.status_code, "ms": elapsed}

    for endpoint in endpoints:
        client.get(endpoint, params=filters).raise_for_status()
    with ThreadPoolExecutor(max_workers=5) as pool:
        samples = list(pool.map(request, endpoints * 50))
    stats = {}
    for endpoint in endpoints:
        rows = [row for row in samples if row["endpoint"] == endpoint]
        times = sorted(row["ms"] for row in rows)
        stats[endpoint] = {
            "requests": len(times),
            "median_ms": round(statistics.median(times), 3),
            "p95_ms": round(times[math.ceil(.95 * len(times)) - 1], 3),
            "max_ms": round(max(times), 3),
            "at_or_above_500_ms": sum(value >= 500 for value in times),
            "non_200": sum(row["status"] != 200 for row in rows),
        }
    return stats, samples


def main():
    parser = argparse.ArgumentParser(description=__doc__)
    parser.add_argument("--db", type=Path, default=Path("var/settlement.db"))
    parser.add_argument("--output", type=Path, default=Path("evidence/01_test_results"))
    args = parser.parse_args()
    args.output.mkdir(parents=True, exist_ok=True)
    db_uri = args.db.resolve().as_uri() + "?mode=ro"
    with closing(sqlite3.connect(db_uri, uri=True)) as db:
        db.row_factory = sqlite3.Row
        filters, expected, exceptions = reconcile(db)
        batch_digests = [row[0] for row in db.execute("SELECT digest FROM batch ORDER BY id")]
        fk_errors = len(db.execute("PRAGMA foreign_key_check").fetchall())
    with socket.socket() as sock:
        sock.bind(("127.0.0.1", 0))
        port = sock.getsockname()[1]
    token = secrets.token_urlsafe(32)
    env = {**os.environ, "SETTLEMENT_API_TOKEN": token, "SETTLEMENT_DB": str(args.db.resolve())}
    server = subprocess.Popen(
        [sys.executable, "-m", "uvicorn", "settlement.api:app", "--host", "127.0.0.1",
         "--port", str(port), "--no-access-log"], env=env,
    )
    endpoints = ["/api/v1/settlement-summary", "/api/v1/merchant-exceptions"]
    try:
        with httpx.Client(base_url=f"http://127.0.0.1:{port}", trust_env=False, timeout=5,
                          headers={"Authorization": f"Bearer {token}"}) as client:
            for _ in range(100):
                if server.poll() is not None:
                    raise RuntimeError("Uvicorn exited before becoming ready")
                try:
                    health = client.get("/health")
                except httpx.ConnectError:
                    time.sleep(.1)
                    continue
                health.raise_for_status()
                break
            else:
                raise RuntimeError("Uvicorn did not become ready")
            summary, merchant_list = [client.get(path, params=filters) for path in endpoints]
            summary.raise_for_status()
            merchant_list.raise_for_status()
```

```
            actual = summary.json()
            actual_exceptions = sorted(row["merchant_id"] for row in merchant_list.json())
            passed = actual == expected and actual_exceptions == exceptions and fk_errors == 0
            metadata = {"captured_at_utc": datetime.now(timezone.utc).isoformat(),
                        "version": "Blue V1 local baseline", "scope": filters,
                        "api_source_sha256":
    hashlib.sha256(Path("settlement/api.py").read_bytes()).hexdigest(),
                        "batch_sha256": batch_digests}
            save(args.output, "kpi-baseline.json", {
                **metadata, "silver_expected": expected, "api_actual": actual,
                "expected_exception_ids": exceptions, "api_exception_ids": actual_exceptions,
                "foreign_key_errors": fk_errors, "health_status": health.status_code,
                "passed": passed,
            })
            stats, samples = measure(client, endpoints, filters)
            timing_passed = all(not row["at_or_above_500_ms"] and not row["non_200"]
                                for row in stats.values())
            save(args.output, "api-timing.json", {
                **metadata, "environment": platform.platform(), "python": platform.python_version(),
                "transport": "Local HTTP against one Uvicorn process", "concurrency": 5,
                "request_count": len(samples), "threshold_ms": 500, "warmup_per_endpoint": 1,
                "passed": timing_passed, "results": stats, "samples": samples,
            })
            print(json.dumps({"reconciled": passed, "metrics": actual, "timing": stats}, indent=2))
            return 0 if passed and timing_passed else 1
    finally:
        server.terminate()
        server.wait(timeout=10)


if __name__ == "__main__":
    raise SystemExit(main())
```

scripts/deploy_ci.sh

```
#!/bin/sh
set -eu

case "${1:-}" in
    development) port=8081 ;;
    production) port=8082 ;;
    *) echo "Choose development or production" >&2; exit 2 ;;
esac

: "${RELEASE_IMAGE:?Missing verified image digest}"
: "${SETTLEMENT_API_TOKEN:?Configure an environment-scoped GitLab secret}"
case "$RELEASE_IMAGE" in
    "$CI_REGISTRY_IMAGE"@sha256:*) ;;
    *) echo "Expected a digest from the project registry" >&2; exit 1 ;;
esac

name="capstone-$1"
volume="$name-data"
candidate="$name-check-$CI_JOB_ID"
backup="$name-backup-$CI_JOB_ID"
docker volume inspect "$volume" >/dev/null
docker pull "$RELEASE_IMAGE"

cleanup_candidate() {
    docker rm -f "$candidate" >/dev/null 2>&1 || true
}

start_app() {
    app_name="$1"
    shift
    docker run -d --name "$app_name" --restart unless-stopped \
        -e SETTLEMENT_API_TOKEN --mount "type=volume,source=$volume,target=/data" \
        "$@" "$RELEASE_IMAGE"
}

trap cleanup_candidate EXIT
start_app "$candidate"
docker exec -i "$candidate" python - < scripts/smoke_ci.py
cleanup_candidate

if docker container inspect "$name" >/dev/null 2>&1; then
    docker rename "$name" "$backup"
    if ! docker stop "$backup"; then
        docker rename "$backup" "$name"
        exit 1
    fi
fi

if start_app "$name" -p "127.0.0.1:$port:8000" &&
    docker exec -i "$name" python - < scripts/smoke_ci.py; then
    if docker container inspect "$backup" >/dev/null 2>&1; then
        docker rm "$backup"
    fi
else
    docker rm -f "$name" >/dev/null 2>&1 || true
    if docker container inspect "$backup" >/dev/null 2>&1; then
        docker rename "$backup" "$name"
        docker start "$name"
    fi
    exit 1
fi
```

scripts/smoke_ci.py

```
"""Check a deployed container over HTTP and emit evidence without credentials."""

import json
import os
import time
from decimal import Decimal
from urllib.error import URLError
from urllib.parse import urlencode
from urllib.request import Request, urlopen


def get(path, params=None):
    url = "http://127.0.0.1:8000" + path
    if params:
        url += "?" + urlencode(params)
    request = Request(url, headers={
        "Authorization": "Bearer " + os.environ["SETTLEMENT_API_TOKEN"],
    })
    with urlopen(request, timeout=3) as response:
        if response.status != 200:
            raise RuntimeError(f"{path} returned HTTP {response.status}")
        return json.load(response, parse_float=Decimal)


def main():
    for attempt in range(30):
        try:
            health = get("/health")
            break
        except (URLError, TimeoutError):
            if attempt == 29:
                raise
            time.sleep(1)
    if health != {"status": "ok"}:
        raise RuntimeError("Unexpected health response")

    status = get("/api/v1/data-status")
    scope = {key: status[key] for key in ("start_date", "end_date")}
    if not all(scope.values()) or not status["last_ingested_at_utc"]:
        raise RuntimeError("Deployment database has no ingested reporting data")
    summary = get("/api/v1/settlement-summary", scope)
    exceptions = get("/api/v1/merchant-exceptions", scope)
    paid = summary["transaction_amount"]
    settled = summary["settled_amount"]
    if not (summary["transaction_count"] > 0 and paid > 0 and 0 <= settled <= paid
            and paid - settled == summary["settlement_gap"]
            and abs(summary["settlement_rate"] - 100 * settled / paid) <= Decimal("0.005")
            and isinstance(exceptions, list)):
        raise RuntimeError("Settlement KPI reconciliation failed")
    print(json.dumps({"health": health, "scope": scope, "summary": summary,
                      "merchant_exception_count": len(exceptions)}, default=str, indent=2))


if __name__ == "__main__":
    main()
```

settlement/__init__.py

settlement/api.py

```
import os
import re
import secrets
import sqlite3
from contextlib import closing
from datetime import date, datetime
from pathlib import Path
from typing import Annotated
from zoneinfo import ZoneInfo

from fastapi import Depends, FastAPI, HTTPException, Query, Request
from fastapi.responses import FileResponse
from fastapi.security import HTTPAuthorizationCredentials, HTTPBearer
from fastapi.staticfiles import StaticFiles
from pydantic import BaseModel, Field

from .db import connect, database_path


app = FastAPI(title="Merchant Settlement Intelligence", version="1.0.0")
static = Path(__file__).with_name("web")
app.mount("/assets", StaticFiles(directory=static), name="assets")
bearer = HTTPBearer(auto_error=False)


def authorize(credentials: Annotated[HTTPAuthorizationCredentials | None, Depends(bearer)]):
    expected = os.getenv("SETTLEMENT_API_TOKEN")
    if not expected:
        raise HTTPException(503, "API access is not configured")
    if not credentials or not secrets.compare_digest(credentials.credentials.encode(),
    expected.encode()):
        raise HTTPException(401, "A valid operations token is required",
                            headers={"WWW-Authenticate": "Bearer"})


def read_db():
    try:
        db = connect(database_path())
    except (OSError, sqlite3.Error):
        raise HTTPException(503, "Settlement database is unavailable") from None
    with closing(db), db:
        db.execute("PRAGMA query_only=ON")
        db.execute("BEGIN")
        yield db


Database = Annotated[sqlite3.Connection, Depends(read_db)]
access = [Depends(authorize)]
# Query fragments below are fixed SQL. Request values are always bound parameters.
MERCHANT_ID = r"^[A-Za-z0-9_-]+$"
AGG = """COUNT(*) AS n, COALESCE(SUM(amount),0) AS amount,
         COALESCE(SUM(credited),0) AS credited, COALESCE(SUM(actual),0) AS actual,
         COUNT(completion) AS completed, COALESCE(SUM(within_sla),0) AS on_time"""


def filters(
    db: Database,
    start_date: Annotated[str, Query()],
    end_date: Annotated[str, Query()],
    merchant_id: Annotated[str | None, Query(pattern=MERCHANT_ID, max_length=64)] = None,
):
    try:
        if not all(re.fullmatch(r"\d{4}-\d{2}-\d{2}", value) for value in (start_date, end_date)):
            raise ValueError
        if date.fromisoformat(start_date) > date.fromisoformat(end_date):
            raise ValueError
    except ValueError:
        raise HTTPException(400, "Use valid YYYY-MM-DD dates with start_date <= end_date") from None
    sql, params = "day BETWEEN ? AND ?", (start_date, end_date)
    if merchant_id:
```

```
        if not db.execute("SELECT 1 FROM merchant WHERE merchant_id=?", (merchant_id,)).fetchone():
            raise HTTPException(404, "Unknown merchant")
        sql += " AND merchant_id=?"
        params += (merchant_id,)
    return sql, params


Filters = Annotated[tuple, Depends(filters)]


class Summary(BaseModel):
    transaction_count: int
    transaction_amount: float
    settled_amount: float
    settlement_gap: float
    settlement_rate: float | None = Field(ge=0, le=100)
    sla_rate: float | None = Field(ge=0, le=100)
    merchant_risk_count: int
    excess_settled_amount: float


class MerchantMetric(BaseModel):
    merchant_id: str
    merchant_name: str
    risk_level: str
    settlement_rate: float
    sla_rate: float | None
    settlement_gap: float


def percentage(numerator, denominator):
    return round(100 * numerator / denominator, 2) if denominator else None


def merchant_rows(db, scope):
    where, params = scope
    return db.execute(f"""
        WITH selected AS (
            SELECT *, ROW_NUMBER() OVER(PARTITION BY merchant_id ORDER BY ts DESC,id DESC) AS latest
            FROM gold WHERE {where}
        )
        SELECT merchant_id, MAX(CASE WHEN latest=1 THEN merchant_name END) AS name,
               MAX(CASE WHEN latest=1 THEN risk END) AS risk, SUM(amount-credited) AS gap, {AGG}
        FROM selected GROUP BY merchant_id ORDER BY gap DESC, merchant_id
    """, params).fetchall()  # nosec B608


def below_rate(row):
    return row["credited"] * 100 < row["amount"] * 95


def below_sla(row):
    return bool(row["completed"] and row["on_time"] * 100 < row["completed"] * 90)


def merchant_metric(row):
    return MerchantMetric(merchant_id=row["merchant_id"], merchant_name=row["name"],
                          risk_level=row["risk"], settlement_rate=percentage(row["credited"],
    row["amount"]),
                          sla_rate=percentage(row["on_time"], row["completed"]),
                          settlement_gap=(row["amount"] - row["credited"]) / 100)


@app.middleware("http")
async def response_headers(request: Request, call_next):
    response = await call_next(request)
    response.headers["X-Content-Type-Options"] = "nosniff"
    if request.url.path.startswith("/api/"):
        response.headers["Cache-Control"] = "no-store"
    return response


@app.get("/", include_in_schema=False)
```

```
def dashboard():
    return FileResponse(static / "index.html")


@app.get("/health")
def health(db: Database):
    try:
        db.execute("SELECT id FROM gold LIMIT 1")
        db.execute("SELECT id FROM batch LIMIT 1")
    except sqlite3.Error:
        raise HTTPException(503, "Settlement schema is unavailable") from None
    return {"status": "ok"}


@app.get("/api/v1/data-status", dependencies=access)
def data_status(db: Database):
    data = db.execute("SELECT MIN(day) AS start_date, MAX(day) AS end_date FROM gold").fetchone()
    return {**dict(data), "last_ingested_at_utc": db.execute("SELECT MAX(loaded_at) FROM
    batch").fetchone()[0]}


@app.get("/api/v1/settlement-summary", dependencies=access, response_model=Summary)
def summary(db: Database, scope: Filters):
    where, params = scope
    row = db.execute(f"SELECT {AGG} FROM gold WHERE {where}", params).fetchone()  # nosec B608
    return Summary(
        transaction_count=row["n"], transaction_amount=row["amount"] / 100,
        settled_amount=row["credited"] / 100, settlement_gap=(row["amount"] - row["credited"]) / 100,
        settlement_rate=percentage(row["credited"], row["amount"]),
        sla_rate=percentage(row["on_time"], row["completed"]),
        merchant_risk_count=sum(below_rate(m) and below_sla(m) for m in merchant_rows(db, scope)),
        excess_settled_amount=(row["actual"] - row["credited"]) / 100,
    )


@app.get("/api/v1/daily-trend", dependencies=access)
def trend(db: Database, scope: Filters):
    where, params = scope
    return [
        {"date": r["day"], "transaction_amount": r["amount"] / 100, "settled_amount": r["credited"] /
    100}
        for r in db.execute(f"SELECT day, {AGG} FROM gold WHERE {where} GROUP BY day ORDER BY day",
    params)  # nosec B608
    ]


@app.get("/api/v1/merchant-gaps", dependencies=access, response_model=list[MerchantMetric])
def merchant_gaps(db: Database, scope: Filters):
    return [merchant_metric(row) for row in merchant_rows(db, scope)[:10]
            if row["amount"] > row["credited"]]


@app.get("/api/v1/merchant-exceptions", dependencies=access, response_model=list[MerchantMetric])
def merchant_exceptions(db: Database, scope: Filters):
    return [merchant_metric(row) for row in merchant_rows(db, scope) if below_rate(row) or
    below_sla(row)]


@app.get("/api/v1/payment-exceptions", dependencies=access)
def payment_exceptions(db: Database, scope: Filters,
                       limit: Annotated[int, Query(ge=1, le=100)] = 25,
                       offset: Annotated[int, Query(ge=0, le=1000000)] = 0):
    where, params = scope
    now = datetime.now(ZoneInfo("Asia/Kolkata")).strftime("%Y-%m-%d %H:%M:%S")
    rows = db.execute(f"""
        SELECT id, merchant_id, ts, risk, amount, credited, completion,
               CASE WHEN completion IS NULL AND unixepoch(?) - unixepoch(ts) > 1800
                    THEN 1 ELSE 0 END AS overdue
        FROM gold WHERE {where} AND (credited < amount OR within_sla=0)
        ORDER BY amount-credited DESC, ts DESC,id LIMIT ? OFFSET ?
    """, (now, *params, limit, offset))  # nosec B608
    return [{
        "transaction_id": row["id"], "merchant_id": row["merchant_id"],
```

```
        "transaction_ts": row["ts"], "risk_level": row["risk"],
        "transaction_amount": row["amount"] / 100, "settled_amount": row["credited"] / 100,
        "settlement_gap": (row["amount"] - row["credited"]) / 100,
        "status": ("SETTLED_LATE" if row["completion"] else "PARTIAL" if row["credited"] else
    "UNSETTLED"),
        "overdue": bool(row["overdue"]),
    } for row in rows]
```

settlement/db.py

```
import os
import sqlite3
from pathlib import Path


def database_path():
    return Path(os.environ.get("SETTLEMENT_DB", "var/settlement.db"))


def connect(path, *, initialize=False):
    path = Path(path)
    if initialize:
        path.parent.mkdir(parents=True, exist_ok=True)
    elif not path.is_file():
        raise FileNotFoundError("Run the settlement ingestion command first")
    db = sqlite3.connect(path, timeout=30, check_same_thread=False)
    db.row_factory = sqlite3.Row
    db.execute("PRAGMA foreign_keys=ON")
    if initialize:
        db.execute("PRAGMA journal_mode=WAL")
        db.executescript(Path(__file__).with_name("schema.sql").read_text())
    return db
```

settlement/ingest.py

```
import argparse
import csv
import hashlib
import io
import json
import re
from contextlib import closing
from datetime import date, datetime
from decimal import Decimal, InvalidOperation
from pathlib import Path
from time import monotonic

from .db import connect, database_path


FIELDS = {
    "merchant": "merchant_id merchant_name merchant_category country risk_level effective_from
    effective_to",
    "transactions": "transaction_id merchant_id customer_id transaction_ts amount currency status
    payment_channel",
    "settlements": "settlement_id transaction_id settlement_ts settlement_amount settlement_status
    settlement_batch",
    "payment_events": "event_id transaction_id event_type event_ts ingestion_ts processing_ms",
}


class RowIssue(Exception):
    def __init__(self, reason, disposition="reject", parent=None):
        self.reason = reason
        self.disposition = disposition
        self.parent = parent


def required(value):
    if not value:
        raise RowIssue("missing_required_field")
    return value


def choice(value, options):
    if value not in options.split():
        raise RowIssue("invalid_enum")
    return value


def money(value, *, positive=False):
    number = Decimal(value)
    if not number.is_finite() or not 0 <= number <= Decimal("100000000000"):
        raise RowIssue("invalid_amount")
    paise = number * 100
    if paise != int(paise) or (positive and not paise):
        raise RowIssue("invalid_amount")
    return int(paise)


def timestamp(value):
    return datetime.strptime(value, "%Y-%m-%d %H:%M:%S").isoformat(sep=" ")


def flag(db, raw, kind, reason):
    db.execute("INSERT OR IGNORE INTO finding(raw_id,disposition,reason) VALUES (?,?,?)",
               (raw, kind, reason))


def ingest_merchant(db, raw, row, affected):
    mid = required(row["merchant_id"])
    start = date.fromisoformat(row["effective_from"]).isoformat()
    end = date.fromisoformat(row["effective_to"]).isoformat() if row["effective_to"] else None
    if end and end < start:
        raise RowIssue("invalid_effective_period")
```

```
    values = (mid, start, end, required(row["merchant_name"]), required(row["merchant_category"]),
              required(row["country"]), choice(row["risk_level"], "LOW MEDIUM HIGH"))
    overlaps = db.execute(
        """SELECT * FROM merchant WHERE merchant_id=? AND effective_from<=COALESCE(?,'9999-12-31')
           AND COALESCE(effective_to,'9999-12-31')>=?""", (mid, end, start)
    ).fetchall()
    if overlaps:
        if len(overlaps) == 1 and tuple(overlaps[0]) == values:
            return
        # A snapshot may close a previously open version before adding a new one.
        prior = overlaps[0]
        if (len(overlaps) == 1 and prior["effective_from"] == start
                and prior["effective_to"] is None and end and tuple(prior)[3:] == values[3:]):
            # Preserve the validity of already accepted facts.
            referenced = db.execute(
                "SELECT 1 FROM payment WHERE merchant_id=? AND merchant_from=? AND substr(ts,1,10)>?",
                (mid, start, end),
            ).fetchone()
            if not referenced:
                db.execute("UPDATE merchant SET effective_to=? WHERE merchant_id=? AND
    effective_from=?",
                           (end, mid, start))
                return
        raise RowIssue("overlapping_merchant_history", "quarantine")
    db.execute("INSERT INTO merchant VALUES (?,?,?,?,?,?,?)", values)
    retry_children(db, "merchant:" + mid, affected)


def insert_fact(db, table, values, raw):
    # Allow only internal table names and bind all CSV values.
    if table not in ('payment', 'settlement', 'event'):
        raise RuntimeError('Unknown fact table')
    old = db.execute(f"SELECT * FROM {table} WHERE id=?", (values[0],)).fetchone()  # nosec B608
    if old:
        if tuple(old)[:-1] == values:
            if table == "event":
                flag(db, raw, "warning", "duplicate_event")
            return False
        if (table == "settlement" and old["status"] == "PENDING"
                and values[4] in ("SETTLED", "FAILED") and old["transaction_id"] == values[1]):
            db.execute(
                "UPDATE settlement SET ts=?,amount=?,status=?,batch_code=?,raw_id=? WHERE id=?",
                (*values[2:], raw, values[0]),
            )
            return True
        raise RowIssue("conflicting_id", "quarantine")
    marks = ",".join("?" for _ in (*values, raw))
    db.execute(f"INSERT INTO {table} VALUES ({marks})", (*values, raw))  # nosec B608
    return True


def process(db, source, raw, row, affected):
    if None in row or any(value is None for value in row.values()):
        raise RowIssue("malformed_csv_row")
    if source == "merchant":
        ingest_merchant(db, raw, row, affected)
        return
    tid = required(row["transaction_id"])
    if source == "transactions":
        ts = timestamp(row["transaction_ts"])
        mid = row["merchant_id"]
        amount = money(row["amount"], positive=True)
        if row["currency"] != "INR":
            raise RowIssue("unsupported_currency")
        status = choice(row["status"], "SUCCESS FAILED REVERSED")
        channel = choice(row["payment_channel"], "POS ONLINE QR")
        if not mid:
            raise RowIssue("missing_merchant", "quarantine")
        version = db.execute(
            """SELECT effective_from FROM merchant WHERE merchant_id=? AND effective_from<=?
               AND (effective_to IS NULL OR effective_to>=?)""", (mid, ts[:10], ts[:10])
        ).fetchone()
        if not version:
```

```
            raise RowIssue("merchant_version_not_found", "quarantine", "merchant:" + mid)
        if insert_fact(db, "payment", (tid, mid, version[0], ts, amount, status, channel), raw):
            affected.add(tid)
            retry_children(db, "payment:" + tid, affected)
        return

    if source == "settlements":
        sid = required(row["settlement_id"])
        status = choice(row["settlement_status"], "SETTLED PENDING FAILED")
        ts = timestamp(row["settlement_ts"]) if row["settlement_ts"] or status == "SETTLED" else None
        values = (sid, tid, ts, money(row["settlement_amount"]), status,
    required(row["settlement_batch"]))
    else:
        eid = required(row["event_id"])
        ts = timestamp(row["event_ts"])
        received = timestamp(row["ingestion_ts"])
        elapsed = (datetime.fromisoformat(received) - datetime.fromisoformat(ts)).total_seconds()
        duration = int(row["processing_ms"])
        if elapsed < 0 or not 0 <= duration <= 2**63 - 1:
            raise RowIssue("invalid_event_timing")
        values = (eid, tid, choice(row["event_type"], "CREATED AUTHORIZED SETTLED FAILED"),
                  ts, received, duration)

    parent = db.execute("SELECT ts FROM payment WHERE id=?", (tid,)).fetchone()
    if not parent:
        raise RowIssue("unmatched_transaction", "quarantine", "payment:" + tid)
    if source == "settlements":
        if ts and ts < parent[0]:
            raise RowIssue("settlement_before_payment")
        if insert_fact(db, "settlement", values, raw):
            affected.add(tid)
    elif insert_fact(db, "event", values, raw) and elapsed > 1800:
        flag(db, raw, "warning", "late_event")


def apply_row(db, source, raw, row, affected):
    try:
        process(db, source, raw, row, affected)
    except (ValueError, InvalidOperation):
        flag(db, raw, "reject", "invalid_field_format")
    except RowIssue as issue:
        flag(db, raw, issue.disposition, issue.reason)
        if issue.parent:
            db.execute("INSERT OR REPLACE INTO retry VALUES (?,?)", (raw, issue.parent))


def retry_children(db, parent, affected):
    rows = db.execute(
        "SELECT b.* FROM retry r JOIN bronze b ON b.id=r.raw_id WHERE r.parent=? ORDER BY b.id",
    (parent,)
    ).fetchall()
    for row in rows:
        db.execute("DELETE FROM retry WHERE raw_id=?", (row["id"],))
        db.execute("UPDATE finding SET resolved=1 WHERE raw_id=? AND disposition='quarantine'",
    (row["id"],))
        apply_row(db, row["source"], row["id"], json.loads(row["payload"]), affected)


def refresh_gold(db, tid):
    payment = db.execute(
        """SELECT p.*, m.name, m.risk FROM payment p JOIN merchant m
           ON m.merchant_id=p.merchant_id AND m.effective_from=p.merchant_from WHERE p.id=?""", (tid,)
    ).fetchone()
    if payment["status"] != "SUCCESS":
        return
    amount = payment["amount"]
    total, completion = 0, None
    for row in db.execute(
        "SELECT * FROM settlement WHERE transaction_id=? AND status='SETTLED' ORDER BY ts,id", (tid,)
    ):
        total += row["amount"]
        if completion is None and total >= amount:
            completion = row["ts"]
```

```
        if total > amount and row["amount"]:
            flag(db, row["raw_id"], "warning", "excess_settlement")
    on_time = bool(completion and
                   (datetime.fromisoformat(completion) -
    datetime.fromisoformat(payment["ts"])).total_seconds() <= 1800)
    db.execute("INSERT OR REPLACE INTO gold VALUES (?,?,?,?,?,?,?,?,?,?,?)",
               (tid, payment["ts"][:10], payment["ts"], payment["merchant_id"], payment["name"],
                payment["risk"], amount, total, min(amount, total), completion, int(on_time)))


def batch_files(folder):
    candidates = {}
    suffixes = set()
    for source in FIELDS:
        matches = [path for path in folder.glob("*.csv")
                   if re.fullmatch(rf"{source}(_\d{{4}}-\d{{2}}-\d{{2}})?\.csv", path.name)]
        if len(matches) != 1:
            raise ValueError(f"Expected one CSV for {source}")
        path = matches[0]
        suffixes.add(path.stem[len(source):])
        candidates[source] = path.read_bytes()
    if len(suffixes) != 1:
        raise ValueError("All four filenames must use the same batch date")
    return candidates


def ingest(folder, path):
    started = monotonic()
    files = batch_files(Path(folder))
    digest = hashlib.sha256(b"".join(hashlib.sha256(data).digest() for data in
    files.values())).hexdigest()
    affected = set()
    with closing(connect(path, initialize=True)) as db, db:
        db.execute("BEGIN IMMEDIATE")
        existing = db.execute("SELECT id FROM batch WHERE digest=?", (digest,)).fetchone()
        if existing:
            return {"batch": existing[0], "replayed": True, "affected_payments": 0}
        batch = db.execute("INSERT INTO batch(digest) VALUES (?)", (digest,)).lastrowid
        counts = {}
        for source, data in files.items():
            reader = csv.DictReader(io.StringIO(data.decode("utf-8-sig"), newline=""), strict=True)
            if reader.fieldnames != FIELDS[source].split():
                raise ValueError(f"Unexpected header for {source}")
            counts[source] = 0
            for line, row in enumerate(reader, 2):
                raw = db.execute("INSERT INTO bronze(batch_id,source,line,payload) VALUES (?,?,?,?)",
                                 (batch, source, line, json.dumps(row))).lastrowid
                apply_row(db, source, raw, row, affected)
                counts[source] += 1
        for tid in sorted(affected):
            refresh_gold(db, tid)
        findings = dict(db.execute(
            """SELECT disposition, count(*) FROM finding f JOIN bronze b ON b.id=f.raw_id
               WHERE b.batch_id=? AND f.resolved=0 GROUP BY disposition""", (batch,)
        ))
        return {"batch": batch, "replayed": False, "rows": counts, "findings": findings,
                "affected_payments": len(affected), "elapsed_seconds": round(monotonic() - started, 2)}


def main():
    parser = argparse.ArgumentParser(description="Load a merchant settlement CSV batch")
    parser.add_argument("folder", type=Path)
    parser.add_argument("--db", type=Path, default=database_path())
    args = parser.parse_args()
    try:
        print(json.dumps(ingest(args.folder, args.db), indent=2))
    except (ValueError, OSError, csv.Error) as exc:
        parser.exit(1, f"Ingestion failed: {exc}\n")


if __name__ == "__main__":
    main()
```

settlement/schema.sql

```
CREATE TABLE IF NOT EXISTS batch (
 id INTEGER PRIMARY KEY, digest TEXT UNIQUE NOT NULL, loaded_at TEXT DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE IF NOT EXISTS bronze (
 id INTEGER PRIMARY KEY, batch_id INTEGER NOT NULL REFERENCES batch(id),
 source TEXT NOT NULL, line INTEGER NOT NULL, payload TEXT NOT NULL, UNIQUE(batch_id,source,line)
);
CREATE TABLE IF NOT EXISTS finding (
 raw_id INTEGER NOT NULL REFERENCES bronze(id), disposition TEXT NOT NULL
 CHECK(disposition IN ('reject','quarantine','warning')), reason TEXT NOT NULL,
 resolved INTEGER NOT NULL DEFAULT 0, PRIMARY KEY(raw_id,reason)
);
CREATE TABLE IF NOT EXISTS retry (
 raw_id INTEGER PRIMARY KEY REFERENCES bronze(id), parent TEXT NOT NULL
);
CREATE INDEX IF NOT EXISTS retry_parent ON retry(parent);
CREATE TABLE IF NOT EXISTS merchant (
 merchant_id TEXT NOT NULL, effective_from TEXT NOT NULL, effective_to TEXT,
 name TEXT NOT NULL, category TEXT NOT NULL, country TEXT NOT NULL,
 risk TEXT NOT NULL CHECK(risk IN ('LOW','MEDIUM','HIGH')),
 CHECK(effective_to IS NULL OR effective_to >= effective_from),
 PRIMARY KEY(merchant_id,effective_from)
);
CREATE TABLE IF NOT EXISTS payment (
 id TEXT PRIMARY KEY NOT NULL, merchant_id TEXT NOT NULL, merchant_from TEXT NOT NULL,
 ts TEXT NOT NULL, amount INTEGER NOT NULL CHECK(amount > 0),
 status TEXT NOT NULL CHECK(status IN ('SUCCESS','FAILED','REVERSED')),
 channel TEXT NOT NULL CHECK(channel IN ('POS','ONLINE','QR')),
 raw_id INTEGER NOT NULL REFERENCES bronze(id),
 FOREIGN KEY(merchant_id,merchant_from) REFERENCES merchant(merchant_id,effective_from)
);
CREATE INDEX IF NOT EXISTS payment_merchant ON payment(merchant_id,ts);
CREATE TABLE IF NOT EXISTS settlement (
 id TEXT PRIMARY KEY NOT NULL, transaction_id TEXT NOT NULL REFERENCES payment(id),
 ts TEXT, amount INTEGER NOT NULL CHECK(amount >= 0),
 status TEXT NOT NULL CHECK(status IN ('SETTLED','PENDING','FAILED')),
 batch_code TEXT NOT NULL, raw_id INTEGER NOT NULL REFERENCES bronze(id),
 CHECK(status <> 'SETTLED' OR ts IS NOT NULL)
);
CREATE INDEX IF NOT EXISTS settlement_payment ON settlement(transaction_id,ts,id);
CREATE TABLE IF NOT EXISTS event (
 id TEXT PRIMARY KEY NOT NULL, transaction_id TEXT NOT NULL REFERENCES payment(id),
 type TEXT NOT NULL CHECK(type IN ('CREATED','AUTHORIZED','SETTLED','FAILED')),
 ts TEXT NOT NULL, received TEXT NOT NULL, processing_ms INTEGER NOT NULL CHECK(processing_ms >= 0),
 raw_id INTEGER NOT NULL REFERENCES bronze(id)
);
CREATE INDEX IF NOT EXISTS event_payment ON event(transaction_id,ts,id);
CREATE TABLE IF NOT EXISTS gold (
 id TEXT PRIMARY KEY NOT NULL REFERENCES payment(id), day TEXT NOT NULL, ts TEXT NOT NULL,
 merchant_id TEXT NOT NULL, merchant_name TEXT NOT NULL, risk TEXT NOT NULL,
 amount INTEGER NOT NULL CHECK(amount > 0), actual INTEGER NOT NULL,
 credited INTEGER NOT NULL CHECK(credited BETWEEN 0 AND amount),
 completion TEXT, within_sla INTEGER NOT NULL CHECK(within_sla IN (0,1))
);
CREATE INDEX IF NOT EXISTS gold_day ON gold(day);
CREATE INDEX IF NOT EXISTS gold_merchant_day ON gold(merchant_id,day);
```

settlement/web/dashboard.css

```
* { box-sizing: border-box; }
:root { font-family: Inter, "Segoe UI", sans-serif; color: #19283d; background: #f3f6fa; }
body { margin: 0; }
[hidden] { display: none !important; }
.frame { width: min(94%, 1420px); margin: auto; }
.mast { background: #14253d; color: #fff; padding: 17px 0; font-size: .93rem; font-weight: 650; }
.mark { color: #5eead4; margin-right: 7px; }
main { padding-bottom: 3rem; }
.intro { display: flex; justify-content: space-between; align-items: baseline; flex-wrap: wrap; gap: 0
    1rem; margin: 24px 0 10px; }
h1 { font-size: clamp(1.8rem, 3vw, 2.55rem); letter-spacing: -.045em; margin: 0; }
h2 { font-size: 1.05rem; margin: 0; }
.panel, .card { background: white; border: 1px solid #dae3ed; border-radius: 10px; padding: 18px; }
form, .pager { display: flex; align-items: end; gap: 12px; flex-wrap: wrap; }
form label { display: grid; gap: 5px; font-size: .83rem; font-weight: 600; }
input, button { font: inherit; border-radius: 6px; padding: 8px 11px; min-height: 40px; }
input { border: 1px solid #adbcd0; color: inherit; background: white; }
button { cursor: pointer; border: 1px solid #165c54; background: #165c54; color: #fff; font-weight: 650;
     font-size: .85rem; }
button:disabled { opacity: .45; cursor: default; }
.secondary { background: white; color: #165c54; }
.cards { display: grid; grid-template-columns: repeat(auto-fit,minmax(min(100%,180px),1fr)); gap: 12px;
    }
.card span { display: block; font-size: .82rem; color: #43566d; margin-bottom: 8px; }
.card strong { display: block; font-size: clamp(1rem, 2vw, 1.52rem); letter-spacing: -.03em; overflow-
    wrap: anywhere; font-variant-numeric: tabular-nums; }
.charts { display: grid; grid-template-columns: 1.35fr 1fr; gap: 14px; margin-top: 16px; }
.charts section { min-width: 0; }
.muted, #freshness, .legend { color: #43566d; font-size: .82rem; }
.legend span { display: inline-block; width: 10px; height: 10px; margin-right: 4px; border-radius: 2px;
    }
.legend .settled { margin-left: 12px; }
.payment { background: #315db0; }.settled { background: #168675; }
canvas { width: 100%; height: 250px; display: block; }
.bar-row { display: grid; grid-template-columns: minmax(84px,1fr) minmax(34px,1fr) auto; align-items:
    center; gap: 10px; font-size: .78rem; margin-top: 12px; }
.bar-label { overflow-wrap: anywhere; }
.bar-track { height: 9px; background: #e7eff2; border-radius: 4px; }
.bar { height: 100%; background: #168675; border-radius: 4px; }
.table-panel { margin-top: 16px; }
.scroll, .daily-scroll { overflow: auto; }
.daily-scroll { max-height: 210px; }
table { width: 100%; border-collapse: collapse; font-size: .86rem; text-align: left; }
th, td { padding: 10px; border-bottom: 1px solid #e4eaf2; white-space: nowrap; }
th { background: #f2f6fa; }
td button { min-height: auto; padding: 4px 8px; }
.pager { margin-top: 12px; }
#status { color: #43566d; }
#status.error { color: #a12121; }
@media (max-width: 850px) { .charts { grid-template-columns: 1fr; } }
```

settlement/web/dashboard.js

```
const $ = id => document.getElementById(id);
const currency = new Intl.NumberFormat('en-IN', { style: 'currency', currency: 'INR',
    maximumFractionDigits: 2 });
const count = new Intl.NumberFormat('en-IN');
const compact = new Intl.NumberFormat('en-IN', { notation: 'compact', maximumFractionDigits: 1 });
const percentage = n => n === null ? '—' : n.toFixed(2) + '%';
let token = '';
let daily = [];
let query;
let offset = 0;
const pageSize = 25;
let sequence = 0;

async function fetchData(endpoint, params) {
  const suffix = params ? '?' + params.toString() : '';
  const response = await fetch('/api/v1/' + endpoint + suffix, {
    headers: { Authorization: 'Bearer ' + token }
  });
  if (!response.ok) {
    const error = await response.json().catch(() => ({}));
    throw new Error(typeof error.detail === 'string' ? error.detail : 'Check the selected filters and
    try again.');
  }
  return response.json();
}

function status(message, isError = false) {
  $('status').textContent = message;
  $('status').classList.toggle('error', isError);
}

$('access').addEventListener('submit', async event => {
  event.preventDefault();
  const button = event.currentTarget.querySelector('button');
  button.disabled = true;
  token = $('token').value;
  $('token').value = '';
  status('Connecting…');
  try {
    const data = await fetchData('data-status');
    $('start').value = data.start_date ?? '';
    $('end').value = data.end_date ?? '';
    $('access').hidden = true;
    $('filters').hidden = false;
    $('freshness').textContent = data.last_ingested_at_utc
      ? 'Last load: ' + new Date(data.last_ingested_at_utc.replace(' ', 'T') + 'Z').toLocaleString('en-
    IN', { timeZone: 'Asia/Kolkata' }) + ' IST'
      : '';
    if (data.start_date) await refresh();
    else status('No successful payments are available.');
  } catch (error) {
    token = '';
    status(error.message, true);
  } finally {
    button.disabled = false;
  }
});

$('disconnect').addEventListener('click', () => {
  sequence++;
  token = '';
  $('access').hidden = false;
  $('filters').hidden = true;
  $('report').hidden = true;
  $('freshness').textContent = '';
  status('');
  $('token').focus();
});

$('filters').addEventListener('submit', event => {
```

```
  event.preventDefault();
  refresh();
});

async function refresh() {
  const request = ++sequence;
  query = new URLSearchParams({ start_date: $('start').value, end_date: $('end').value });
  if ($('merchant').value) query.set('merchant_id', $('merchant').value);
  $('report').hidden = true;
  status('Loading settlement metrics…');
  try {
    const [summary, days, gaps, merchants, payments] = await Promise.all([
      fetchData('settlement-summary', query), fetchData('daily-trend', query),
      fetchData('merchant-gaps', query), fetchData('merchant-exceptions', query),
      fetchData('payment-exceptions', paymentQuery(0)),
    ]);
    if (request !== sequence) return;
    daily = days;
    renderCards(summary);
    renderGaps(gaps);
    renderMerchants(merchants);
    renderPayments(payments, 0);
    renderDailyValues();
    $('report').hidden = false;
    drawTrend();
    status(summary.transaction_count ? '' : 'No successful payments match these filters.');
  } catch (error) {
    if (request === sequence) status(error.message, true);
  }
}

function renderCards(summary) {
  const metrics = [
    ['Successful payments', count.format(summary.transaction_count)],
    ['Transaction volume', currency.format(summary.transaction_amount)],
    ['Settlement rate', percentage(summary.settlement_rate)],
    ['Settlement gap', currency.format(summary.settlement_gap)],
    ['30-minute SLA', percentage(summary.sla_rate)],
    ['Merchants breaching both thresholds', count.format(summary.merchant_risk_count)],
  ];
  $('cards').replaceChildren(...metrics.map(([label, value]) => {
    const card = document.createElement('article');
    card.className = 'card';
    const title = document.createElement('span');
    const number = document.createElement('strong');
    title.textContent = label;
    number.textContent = value;
    card.append(title, number);
    return card;
  }));
}

function renderGaps(rows) {
  $('gaps').replaceChildren();
  if (!rows.length) $('gaps').textContent = 'No merchant gaps in this period.';
  for (const row of rows) {
    const item = document.createElement('div');
    item.className = 'bar-row';
    const label = document.createElement('span');
    label.className = 'bar-label';
    label.textContent = row.merchant_name;
    const track = document.createElement('div');
    track.className = 'bar-track';
    track.setAttribute('aria-hidden', 'true');
    const bar = document.createElement('div');
    bar.className = 'bar';
    bar.style.width = 100 * row.settlement_gap / rows[0].settlement_gap + '%';
    track.append(bar);
    const value = document.createElement('span');
    value.textContent = currency.format(row.settlement_gap);
    item.append(label, track, value);
    $('gaps').append(item);
  }
```

```
}

function tableRow(values) {
  const row = document.createElement('tr');
  for (const value of values) {
    const cell = row.insertCell();
    if (value instanceof Node) cell.append(value);
    else cell.textContent = value;
  }
  return row;
}

function renderMerchants(rows) {
  $('exceptions').replaceChildren(...rows.map(row => {
    const button = document.createElement('button');
    button.className = 'secondary';
    button.textContent = 'Filter';
    button.setAttribute('aria-label', 'Filter by ' + row.merchant_name);
    button.onclick = () => {
      $('merchant').value = row.merchant_id;
      refresh();
    };
    return tableRow([row.merchant_name + ' · ' + row.merchant_id, row.risk_level,
      percentage(row.settlement_rate), percentage(row.sla_rate), currency.format(row.settlement_gap),
    button]);
  }));
  if (!rows.length) $('exceptions').append(tableRow(['No merchants breached a threshold.']));
}

function paymentQuery(skip) {
  const params = new URLSearchParams(query);
  params.set('limit', pageSize);
  params.set('offset', skip);
  return params;
}

function renderPayments(rows, skip) {
  offset = skip;
  $('payments').replaceChildren(...rows.map(row => tableRow([row.transaction_id, row.merchant_id,
    row.transaction_ts, row.status.replaceAll('_', ' ') + (row.overdue ? ' · overdue' : ''),
    currency.format(row.settlement_gap)])));
  if (!rows.length) $('payments').append(tableRow(['No payment exceptions.']));
  $('prev').disabled = skip === 0;
  $('next').disabled = rows.length < pageSize;
}

async function pageBy(delta) {
  const request = ++sequence;
  const skip = offset + delta;
  $('prev').disabled = $('next').disabled = true;
  try {
    const rows = await fetchData('payment-exceptions', paymentQuery(skip));
    if (request === sequence) renderPayments(rows, skip);
  } catch (error) {
    if (request === sequence) {
      status(error.message, true);
      $('prev').disabled = offset === 0;
      $('next').disabled = false;
    }
  }
}

$('prev').onclick = () => pageBy(-pageSize);
$('next').onclick = () => pageBy(pageSize);

function renderDailyValues() {
  const table = document.createElement('table');
  const header = table.createTHead().insertRow();
  for (const name of ['Payment date', 'Successful payments', 'Settled']) {
    const cell = document.createElement('th');
    cell.textContent = name;
    header.append(cell);
  }
```

```
  const body = table.createTBody();
  for (const row of daily) {
    body.append(tableRow([row.date, currency.format(row.transaction_amount),
    currency.format(row.settled_amount)]));
  }
  $('daily-values').replaceChildren(table);
}

function drawTrend() {
  if ($('report').hidden) return;
  const canvas = $('trend');
  const width = canvas.clientWidth;
  const height = canvas.clientHeight;
  const scale = window.devicePixelRatio || 1;
  canvas.width = width * scale;
  canvas.height = height * scale;
  const ctx = canvas.getContext('2d');
  ctx.scale(scale, scale);
  ctx.font = '11px Segoe UI, sans-serif';
  ctx.fillStyle = '#43566d';
  if (!daily.length) {
    ctx.fillText('No values in this period.', 6, 22);
    return;
  }
  const max = daily.reduce((value, row) => Math.max(value, row.transaction_amount, row.settled_amount),
    1);
  const left = 60, right = width - 18, top = 15, bottom = height - 29;
  const first = Date.parse(daily[0].date);
  const duration = Date.parse(daily.at(-1).date) - first;
  const x = row => duration ? left + (right - left) * (Date.parse(row.date) - first) / duration : (left
    + right) / 2;
  const y = value => bottom - (bottom - top) * value / max;
  for (let i = 0; i <= 4; i++) {
    const value = max * i / 4;
    ctx.strokeStyle = '#dae3ed';
    ctx.beginPath();
    ctx.moveTo(left, y(value));
    ctx.lineTo(right, y(value));
    ctx.stroke();
    ctx.fillText('₹' + compact.format(value), 2, y(value) + 4);
  }
  const labels = new Set([0, Math.floor((daily.length - 1) / 2), daily.length - 1]);
  for (const i of labels) {
    const label = daily[i].date.slice(5);
    ctx.fillText(label, Math.min(width - 36, x(daily[i]) - 13), height - 7);
  }
  for (const [field, color] of [['transaction_amount', '#315db0'], ['settled_amount', '#168675']]) {
    ctx.strokeStyle = ctx.fillStyle = color;
    ctx.lineWidth = 2;
    ctx.beginPath();
    daily.forEach((row, i) => i ? ctx.lineTo(x(row), y(row[field])) : ctx.moveTo(x(row),
    y(row[field])));
    ctx.stroke();
    if (daily.length === 1) {
      ctx.beginPath();
      ctx.arc(x(daily[0]), y(daily[0][field]), 3, 0, 2 * Math.PI);
      ctx.fill();
    }
  }
}

new ResizeObserver(drawTrend).observe($('trend'));
```

settlement/web/index.html

```
<!doctype html>
<html lang="en">
<head>
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="color-scheme" content="light">
<title>Merchant Settlement Intelligence</title>
<link rel="stylesheet" href="/assets/dashboard.css">
<script src="/assets/dashboard.js" defer></script>
</head>
<body>
<header class="mast"><div class="frame"><span class="mark" aria-hidden="true"> </span> Payments
    Operations</div></header>
<main class="frame">
<div class="intro"><h1>Merchant settlement <small>Version 2</small></h1><p id="freshness"></p></div>
<form id="access" class="panel">
  <label>Operations token <input id="token" type="password" required autocomplete="off"></label>
  <button>Connect</button>
</form>
<form id="filters" class="panel filters" hidden>
  <label>From <input id="start" type="date" required></label>
  <label>To <input id="end" type="date" required></label>
  <label>Merchant ID <input id="merchant" pattern="[A-Za-z0-9_\-]+" maxlength="64" placeholder="All
    merchants"></label>
  <button>Apply filters</button><button class="secondary" id="disconnect"
    type="button">Disconnect</button>
</form>
<p id="status" role="status"></p>
<div id="report" hidden>
<section class="cards" aria-label="Settlement KPIs" id="cards"></section>
<div class="charts">
  <section class="panel"><h2>Daily payment and settlement value</h2><p class="muted">Grouped by payment
    date · IST</p>
  <p class="legend"><span class="payment"></span> Successful payments <span class="settled"></span>
    Settled</p>
  <canvas id="trend" height="250" role="img" aria-label="Daily payment and settlement value"></canvas>
  <details><summary>Daily values</summary><div class="daily-scroll" id="daily-
    values"></div></details></section>
  <section class="panel"><h2>Top 10 merchant gaps</h2><div id="gaps"></div></section>
</div>
<section class="panel table-panel"><h2>Merchant exceptions</h2><p class="muted">Settlement rate below
    95% or SLA below 90%.</p>
<div class="scroll"><table><thead><tr><th>Merchant</th><th>Risk</th><th>Settlement
    rate</th><th>SLA</th><th>Gap</th><th scope="col">Investigate</th></tr></thead><tbody
    id="exceptions"></tbody></table></div>
</section>
<section class="panel table-panel"><h2>Payment exceptions</h2><p class="muted">Outstanding or settled
    after 30 minutes.</p>
<div class="scroll"><table><thead><tr><th>Payment</th><th>Merchant</th><th>Payment time
    (IST)</th><th>Status</th><th>Outstanding</th></tr></thead><tbody
    id="payments"></tbody></table></div>
<div class="pager"><button class="secondary" id="prev">Previous</button><button class="secondary"
    id="next">Next</button></div>
</section>
</div>
</main>
</body>
</html>
```

start.ps1

```
$ErrorActionPreference = 'Stop'
Push-Location $PSScriptRoot
try {
    $python = if (Test-Path '.venv/Scripts/python.exe') { '.venv/Scripts/python.exe' } else { 'python' }
    if (-not $env:SETTLEMENT_API_TOKEN) {
        $env:SETTLEMENT_API_TOKEN = & $python -c "import secrets; print(secrets.token_urlsafe(32))"
        if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
    }
    Write-Host 'Dashboard: http://127.0.0.1:8000'
    Write-Host "Operations token: $env:SETTLEMENT_API_TOKEN"
    & $python -m uvicorn settlement.api:app --host 127.0.0.1 --port 8000 --no-access-log
    if ($LASTEXITCODE -ne 0) { exit $LASTEXITCODE }
} finally {
    Pop-Location
}
```

tests/acceptance.feature

```
Feature: Investigate merchant settlement performance
  Scenario: Unsettled successful payment
    Given a successful payment with no completed settlement
    When the pipeline processes it
    Then the payment appears as UNSETTLED with the full amount outstanding

  Scenario: Split settlements
    Given a payment for 10000 with completed settlements for 8000 and 2000
    When reporting totals are calculated
    Then payment volume and settled value are both 10000 and the gap is zero

  Scenario: Settlement SLA boundary
    Given a payment becomes fully settled exactly 30 minutes after payment
    When the SLA is calculated
    Then the completion meets the SLA

  Scenario: Historical merchant risk
    Given a merchant was LOW in February and HIGH in April
    When reporting a February payment
    Then the payment uses LOW risk

  Scenario: Late and out-of-order events
    Given an earlier lifecycle event arrives after a later one
    When the batch is processed
    Then business order uses event time and arrival lag uses ingestion time

  Scenario: Incremental replay
    Given a batch adds a settlement for an existing transaction
    When the same batch is loaded twice
    Then the original payment's totals change once

  Scenario: Invalid and unmatched records
    Given a negative settlement and a settlement with no matching transaction
    When the batch runs
    Then both records remain traceable with their respective dispositions

  Scenario: Merchant thresholds
    Given a merchant has a 94 percent settlement rate and 92 percent SLA
    When merchant performance is reported
    Then it appears in exceptions and is excluded from the Merchant Risk count
```

tests/test_platform.py

```
import csv
from contextlib import closing
from pathlib import Path

import pytest
from fastapi.testclient import TestClient

from settlement.api import app
from settlement.db import connect
from settlement.ingest import FIELDS, ingest


def write_batch(folder, **data):
    folder.mkdir()
    for source, fields in FIELDS.items():
        with (folder / f"{source}.csv").open("w", newline="", encoding="utf-8") as out:
            writer = csv.writer(out)
            writer.writerow(fields.split())
            writer.writerows(data.get(source, []))
    return folder


def payment(tid, merchant="M1", status="SUCCESS", ts="2026-02-01 10:00:00", amount="10000"):
    return (tid, merchant, "private-customer", ts, amount, "INR", status, "QR")


def settlement(sid, tid, amount, minute="30", status="SETTLED"):
    return (sid, tid, "2026-02-01 10:" + minute + ":00", amount, status, "B1")


@pytest.fixture
def dataset(tmp_path):
    event = ("E1", "T1", "CREATED", "2026-02-01 10:00:00", "2026-02-01 10:45:00", 2)
    base = write_batch(
        tmp_path / "base",
        merchant=[
            ("M1", "Test Retail", "Retail", "IN", "LOW", "2026-01-01", "2026-03-31"),
            ("M1", "Test Retail", "Retail", "IN", "HIGH", "2026-04-01", ""),
        ],
        transactions=[payment("T1"), payment("T2", status="FAILED"), payment("T3", merchant=""),
                      payment("T4", ts="2026-04-01 10:00:00"), payment("T5")],
        settlements=[settlement("S1", "T1", "8000", "10"), settlement("S2", "T1", "-3"),
                     settlement("S3", "T6", "10000"), settlement("S4", "T2", "10000")],
        payment_events=[("E2", "T1", "AUTHORIZED", "2026-02-01 10:00:02", "2026-02-01 10:00:04", 2),
                        event, event],
    )
    path = tmp_path / "test.db"
    ingest(base, path)
    return path


def test_silver_grain_quality_and_risk(dataset):
    with closing(connect(dataset)) as db:
        assert db.execute("PRAGMA foreign_key_check").fetchall() == []
        assert db.execute("SELECT id FROM payment GROUP BY id HAVING COUNT(*)>1").fetchall() == []
        assert [tuple(r) for r in db.execute("SELECT id,risk FROM gold ORDER BY id")] == [
            ("T1", "LOW"), ("T4", "HIGH"), ("T5", "LOW"),
        ]
        assert [r[0] for r in db.execute("SELECT id FROM event WHERE transaction_id='T1' ORDER BY
    ts,id")] == [
            "E1", "E2",
        ]
        assert {tuple(r) for r in db.execute("SELECT disposition,reason FROM finding")} == {
            ("warning", "late_event"), ("warning", "duplicate_event"),
            ("reject", "invalid_amount"), ("quarantine", "missing_merchant"),
            ("quarantine", "unmatched_transaction"),
        }


def test_incremental_split_orphan_repair_and_replay(dataset, tmp_path, client):
```

```
    with closing(connect(dataset)) as db:
        assert tuple(db.execute("SELECT amount,credited,completion FROM gold WHERE id='T1'").fetchone())
     == (
            1000000, 800000, None,
        )
    new = write_batch(tmp_path / "next", transactions=[payment("T6")],
                      settlements=[settlement("S5", "T1", "2000")])
    assert ingest(new, dataset)["affected_payments"] == 2
    assert ingest(new, dataset)["affected_payments"] == 0
    with closing(connect(dataset)) as db:
        assert tuple(db.execute("SELECT amount,credited,completion,within_sla FROM gold WHERE
    id='T1'").fetchone()) == (
            1000000, 1000000, "2026-02-01 10:30:00", 1,
        )
        assert db.execute("SELECT credited FROM gold WHERE id='T6'").fetchone()[0] == 1000000
        assert db.execute("SELECT resolved FROM finding WHERE
    reason='unmatched_transaction'").fetchone()[0] == 1
        assert db.execute("SELECT COUNT(*) FROM batch").fetchone()[0] == 2
    response = client.get("/api/v1/settlement-summary", params=scope)
    assert response.status_code == 200
    summary = response.json()
    # T1 has two settlements, T6 has one, and T5 is outstanding.
    assert summary["transaction_count"] == 3
    assert summary["transaction_amount"] == 30000
    assert summary["settled_amount"] == 20000
    assert summary["settlement_gap"] == 10000
    assert summary["settlement_rate"] == 66.67


def test_failed_batch_is_atomic(dataset, tmp_path):
    new = write_batch(tmp_path / "bad", transactions=[payment("NEW")])
    (new / "settlements.csv").write_text("incorrect_header\n", encoding="utf-8")
    with pytest.raises(ValueError, match="Unexpected header"):
        ingest(new, dataset)
    with closing(connect(dataset)) as db:
        assert db.execute("SELECT COUNT(*) FROM batch").fetchone()[0] == 1
        assert db.execute("SELECT id FROM payment WHERE id='NEW'").fetchone() is None


@pytest.fixture
def client(dataset, monkeypatch):
    monkeypatch.setenv("SETTLEMENT_DB", str(dataset))
    monkeypatch.setenv("SETTLEMENT_API_TOKEN", "test-fixture")
    return TestClient(app, headers={"Authorization": "Bearer test-fixture"})


scope = {"start_date": "2026-02-01", "end_date": "2026-02-28"}


def test_api_contract_and_dashboard(client):
    endpoint = "/api/v1/settlement-summary"
    response = client.get(endpoint, params=scope)
    assert response.status_code == 200
    assert response.json() == {
        "transaction_count": 2, "transaction_amount": 20000, "settled_amount": 8000,
        "settlement_gap": 12000, "settlement_rate": 40, "sla_rate": None,
        "merchant_risk_count": 0, "excess_settled_amount": 0,
    }
    assert client.get(endpoint, params={**scope, "start_date": "2026-02-30"}).status_code == 400
    assert client.get(endpoint, params={**scope, "merchant_id": "UNKNOWN"}).status_code == 404
    assert client.get(endpoint, params={**scope, "merchant_id": "' OR 1=1"}).status_code == 422
    assert client.get(endpoint).status_code == 422
    assert client.get(endpoint, params=scope, headers={"Authorization": "Bearer bad"}).status_code ==
    401
    assert client.get("/health").json() == {"status": "ok"}
    assert client.get("/").status_code == 200
    assert client.get("/assets/dashboard.js").status_code == 200
    assert client.get("/openapi.json").status_code == 200
    assert client.get("/api/v1/merchant-exceptions", params=scope).json()[0]["merchant_id"] == "M1"
    assert client.get("/api/v1/merchant-gaps", params=scope).json()[0]["settlement_gap"] == 12000
    assert client.get("/api/v1/daily-trend", params=scope).json() == [
        {"date": "2026-02-01", "transaction_amount": 20000, "settled_amount": 8000},
    ]
```

```
    payments = client.get("/api/v1/payment-exceptions", params=scope)
    assert [(row["transaction_id"], row["status"]) for row in payments.json()] == [
        ("T5", "UNSETTLED"), ("T1", "PARTIAL"),
    ]
    assert "private-customer" not in payments.text


def test_over_settlement_capped_and_late_sla(client, dataset, tmp_path):
    new = write_batch(tmp_path / "excess", settlements=[
        settlement("S6", "T1", "3000", "31"), settlement("S7", "T5", "10000", "31"),
    ])
    ingest(new, dataset)
    result = client.get("/api/v1/settlement-summary", params=scope).json()
    assert result["settlement_rate"] == 100
    assert result["settlement_gap"] == 0
    assert result["excess_settled_amount"] == 1000
    assert result["sla_rate"] == 0
    assert result["merchant_risk_count"] == 0
    assert len(client.get("/api/v1/merchant-exceptions", params=scope).json()) == 1



def test_merchant_gap_ranks_aggregates(client, dataset, tmp_path):
    batch = write_batch(
        tmp_path / "new-merchants",
        merchant=[
            ("M2", "Small gaps", "Retail", "IN", "LOW", "2026-01-01", ""),
            ("M3", "Large gap", "Retail", "IN", "HIGH", "2026-01-01", ""),
        ],
        transactions=[
            payment("T60", merchant="M2", amount="600"),
            payment("T7", merchant="M2", amount="600"),
            payment("T8", merchant="M3", amount="1000"),
            payment("T9", merchant="M3", amount="300"),
        ],
        settlements=[settlement("S6", "T9", "300", "31")],
    )
    ingest(batch, dataset)
    gaps = client.get("/api/v1/merchant-gaps", params=scope).json()
    assert [(row["merchant_id"], row["settlement_gap"]) for row in gaps] == [
        ("M1", 12000), ("M2", 1200), ("M3", 1000),
    ]
    assert client.get("/api/v1/settlement-summary", params=scope).json()["merchant_risk_count"] == 1
```

Appendix C / Evidence and repository

Repository: GitLab capstone_final

Raw source CSV records, credentials, Docker archives and the full scanner dump are referenced through the project and its local artifacts. The evidence below records test scope, observed measurements and incident recovery.

01_test_results/README.md

```
# Day 2 Task 1 — Blue V1 readiness evidence

Captured locally on 12 November 2026 using the Day 1 app and the supplied base and October 1 incremental
     batches. The API source hash and both batch hashes are recorded in the JSON evidence.

| Check | Result | Evidence |
|---|---|---|
| Full automated suite | 6 passed. One deprecation warning from Starlette's test client. | [Test
    log](pytest.log), [JUnit XML](pytest.xml) |
| Split settlements | The regression fixture includes one payment with two completed settlements,
    another fully settled payment, and one outstanding payment. The API returned 3 payments, INR 30,000
    successful volume, INR 20,000 settled, INR 10,000 gap, and 66.67% settlement rate. | [Test
    assertions](../../tests/test_platform.py) |
| Reconciliation and health | Independent SQL against Silver matched the API summary and merchant
    exception IDs. Zero foreign-key errors. Health returned HTTP 200. | [KPI baseline](kpi-
    baseline.json) |
| Live API timing | 100 of 100 HTTP requests returned 200 in under 500 ms. Summary p95 387.308 ms, max
    415.358 ms. Merchant exceptions p95 362.666 ms, max 387.606 ms. | [Timing and raw samples](api-
    timing.json) |

The baseline covers 2026-01-01 through 2026-10-01. It contains 31,763 successful payments, a 94.70%
    settlement rate, an INR 7,246,416.79 gap, and an 82.03% SLA rate.

The timing profile uses five concurrent request workers and one local Uvicorn process. It sends 50
    requests per endpoint across the full date range after one warmup per endpoint. The 500 ms result
    applies to this local workload.

Run from the project root in PowerShell.

```powershell
$results = "evidence/01_test_results"
New-Item -ItemType Directory -Force $results | Out-Null
.venv/Scripts/python.exe -m pytest -v --junitxml="$results/pytest.xml" |
    Tee-Object -FilePath "$results/pytest.log"
.venv/Scripts/python.exe scripts/day2_readiness.py
```
```

02_security_results/README.md

```
# Day 2 Task 2 Security Gate

Scanned the local capstone workspace and built image on 12 November 2026. The four checks are complete
    and the image meets the capstone's zero-critical target.

| Check | Tool and scope | Result |
|---|---|---|
| Static code analysis | Bandit 1.9.4 on the application Python package | Zero unresolved findings. Six
    existing B608 exceptions cover SQL built from fixed internal fragments or an allowlisted table name.
     Request and CSV values are bound parameters. [Report](bandit.json) |
| Secret scan | Gitleaks 8.30.1 on 27 project files | Two redacted generic API key alerts. Both are
    false positives on `api_source_sha256` in Day 2 Task 1 evidence. Each value matched a fresh SHA-256
    hash of `settlement/api.py`. No confirmed secrets. [Report](gitleaks.json) · [File list](secret-
    scan-scope.txt) |
| Dependency scan | pip-audit 2.10.1 on requirements.txt and resolved dependencies | 15 packages checked
     and zero known vulnerabilities reported. [Report](dependencies.json) |
| Container image scan | Trivy vulnerability scan of the local `capstone-settlement:day2-security` image
     | 0 critical, 51 high, 81 medium, 67 low and 19 unknown package findings. [Trivy report](trivy-
    image.json) |

The workspace has no Git metadata. The secret scan covers the listed current files. Git history, the
    virtual environment, runtime files, and local environment files are outside that scan. The two hash
    alerts need no code fix or credential rotation.

Docker's image index was `sha256:eb3a776625cc92f46e74d2c869afe9fd400f52fbc2657bc263bc498d6e1c1eac`. The
    saved archive links that index to the scanned image config
    `sha256:f926d345f80145bb6f84fd541409c0ae377504ef8751287a6a2abfe6f2d109d9`.

Of the 51 high findings, 49 were in Debian packages and two were in Python package metadata inside the
    image. Trivy listed fixes for high findings in `zlib1g`, `jaraco.context` and `wheel`. Remediation
    is to update the affected base packages and Python build tooling to fixed versions or remove unused
    build tooling from the runtime image, then rebuild and rescan. Other high findings had no fix
    version listed in this report and need follow-up when patches are available. These findings remain
    open on the scanned image.

To repeat the scan, build and export the exact local artifact. The archive lets Trivy scan the app image
     without access to the Docker socket.

```powershell
docker build --pull -t capstone-settlement:day2-security .
docker image inspect capstone-settlement:day2-security --format '{{.Id}}'
docker save -o var/day2-security-image.tar capstone-settlement:day2-security
$archive = (Resolve-Path var/day2-security-image.tar).Path
$evidence = (Resolve-Path evidence/02_security_results).Path
docker run --rm --mount "type=bind,source=$archive,target=/image.tar,readonly" `
  --mount "type=bind,source=$evidence,target=/evidence" aquasec/trivy:latest `
  image --input /image.tar --scanners vuln --format json --output /evidence/trivy-image.json
```

Review the Trivy report for severity and fix versions before closing the gate. A completed scan with any
     critical findings requires remediation and a rescan.
```

jenkins/dev-smoke.json


```
{
  "health": {
    "status": "ok"
  },
  "scope": {
    "start_date": "2026-01-01",
    "end_date": "2026-10-01"
  },
  "summary": {
    "transaction_count": 31763,
    "transaction_amount": "136675776.64",
    "settled_amount": "129429359.85",
    "settlement_gap": "7246416.79",
    "settlement_rate": "94.7",
    "sla_rate": "82.03",
    "merchant_risk_count": 16,
    "excess_settled_amount": "0.0"
  },
  "merchant_exception_count": 61
}
```


jenkins/image.json


```
{
    "image":  "capstone-settlement:jenkins-2",
    "image_id":  "sha256:c48ec10b12f9c3649cd539d2ac1b45ea9a78fbf86e7ca1c5da80ea213dd27461",
    "commit":  null,
    "archive_sha256":  "9627C721F79CD3B4F89F0A0FB0A575583276FA8A3B7DF9D51E41DB9843235994",
    "critical_findings":  0
}
```


jenkins/blue-green-cutover.md

```
# Blue-green cutover evidence

On 12 November 2026, the local settlement dashboard switched from Blue V1 (Jenkins build #2, `capstone-
    settlement:jenkins-2`) to Green V2 (Jenkins build #3, `capstone-settlement:jenkins-3`). Green was
    built from commit `53c1ddf`.

Before the switch, the shared URL `http://127.0.0.1:8080/` routed to `blue-v1`. Green passed its direct
    smoke test on port 8083 and its results matched Blue. The Nginx configuration check passed and Nginx
     accepted the reload. After the switch, the same shared URL showed Version 2, and the API responses
    carried the `green-v2` deployment header.

| Check through the shared URL after cutover | HTTP status | Observed time |
| --- | --- | --- |
| `/health` | 200 | 83 ms |
| `/api/v1/settlement-summary` | 200 | 338 ms |
| `/api/v1/merchant-exceptions` | 200 | 161 ms |

All three checks met the 500 ms target in this smoke test. The settlement summary and merchant exception
     count matched the Blue baseline for 2026-01-01 through 2026-10-01.

| Check | Blue V1 and Green V2 |
| --- | --- |
| Health | `{"status":"ok"}` |
| Transaction count | 31,763 |
| Transaction amount | 136,675,776.64 |
| Settled amount | 129,429,359.85 |
| Settlement gap | 7,246,416.79 |
| Settlement rate | 94.70% |
| SLA rate | 82.03% |
| Merchant risk count | 16 |
| Excess settled amount | 0.00 |
| Merchant exception count | 61 |

The transaction amount minus the settled amount equals the reported gap. Blue remained healthy after the
     switch and later served the shared URL during the [incident rollback drill](blue-green-
    incident.md).
```

jenkins/blue-green-incident.md

```
# Green settlement incident and rollback drill

On 12 November 2026, Green V2 was serving the local dashboard on port 8080. Its baseline for 1 January
    through 1 October was a 94.70% settlement rate and zero excess settlements.

We backed up Green's SQLite database and simulated duplicate settlements by doubling the actual settled
    amount for three fully settled payments across three merchants in Green's reporting table. The app
    and Jenkins image stayed the same. The API reported 560,042.46 in excess settlements and continued
    returning HTTP 200. The rate stayed at 94.70% because it uses credited amounts capped at the
    original payment amount.

We routed the shared URL back to Blue V1 at about 17:58 IST. The first probe ran before the Nginx reload
     had taken effect. The subsequent probe confirmed the `blue-v1` response header, HTTP 200, and the
    original settlement summary with zero excess. We captured Green's faulty response through its
    separate preview URL after rollback, restored the three modified amounts from backup, and verified
    that Green's preview returned to the same baseline.

The rollback restored trusted reporting while Green was investigated. A check on excess settlement
    amounts would catch this scenario even when health and settlement rate pass. For a future deployment
     gate, compare excess amounts with the approved baseline and flag unexpected increases.

[API observations and affected counts](blue-green-incident.json)
```

jenkins/blue-green-incident.json


```
{
  "incident": "Simulated duplicate settlements in Green reporting data",
  "scope": {
    "end_date": "2026-10-01",
    "start_date": "2026-01-01"
  },
  "impact": {
    "payments": 3,
    "merchants": 3,
    "excess_minor_units": 56004246
  },
  "green_fault_via_preview": {
    "url": "http://127.0.0.1:8083",
    "observed_at": "2026-11-12T12:29:21.2222379+00:00",
    "route": null,
    "health_http": 200,
    "health": {
      "status": "ok"
    },
    "summary_http": 200,
    "summary": {
      "transaction_count": 31763,
      "transaction_amount": 136675776.64,
      "settled_amount": 129429359.85,
      "settlement_gap": 7246416.79,
      "settlement_rate": 94.7,
      "sla_rate": 82.03,
      "merchant_risk_count": 16,
      "excess_settled_amount": 560042.46
    }
  },
  "blue_recovery_via_shared_url": {
    "url": "http://127.0.0.1:8080",
    "observed_at": "2026-11-12T12:29:21.5506985+00:00",
    "route": [
      "blue-v1"
    ],
    "health_http": 200,
    "health": {
      "status": "ok"
    },
    "summary_http": 200,
    "summary": {
      "transaction_count": 31763,
      "transaction_amount": 136675776.64,
      "settled_amount": 129429359.85,
      "settlement_gap": 7246416.79,
      "settlement_rate": 94.7,
      "sla_rate": 82.03,
      "merchant_risk_count": 16,
      "excess_settled_amount": 0.0
    }
  },
  "green_after_cleanup": {
    "url": "http://127.0.0.1:8083",
    "observed_at": "2026-11-12T12:29:22.1787163+00:00",
    "route": null,
    "health_http": 200,
    "health": {
      "status": "ok"
    },
    "summary_http": 200,
    "summary": {
      "transaction_count": 31763,
      "transaction_amount": 136675776.64,
      "settled_amount": 129429359.85,
      "settlement_gap": 7246416.79,
      "settlement_rate": 94.7,
      "sla_rate": 82.03,
      "merchant_risk_count": 16,
```

```
      "excess_settled_amount": 0.0
    }
  }
}
```